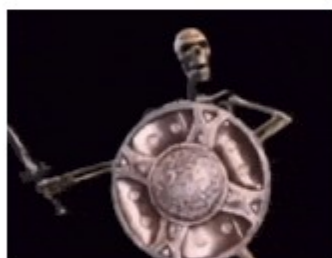
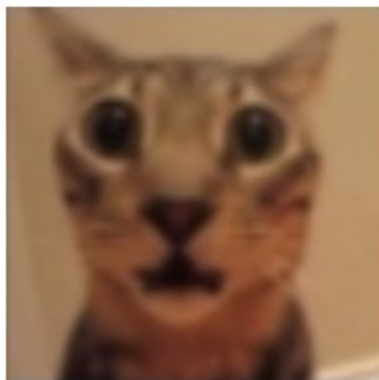
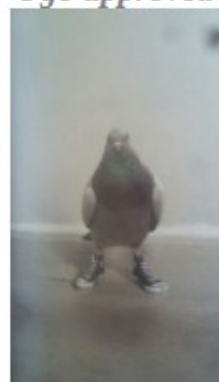


Date confermate in questo PDF:

- 24-01-2024;
- 19-06-2024;
- 20-01-2025;
- 16-09-2025;
- 08-03-2023;
- 03-05-2023;
- 11-05-2025?;
- *Altre sconosciute.*



Ugo approved



10:01 S 📶 📷

5G 📶 77



forms.office.com

**✖ Non corretto** 0/1 Punti

13

Qual è la lunghezza di `a` al termine del seguente frammento di codice?

```
char a[] = "programming";  
char b[] = "algorithm";  
int i;  
  
for(i=0; b[i] != '\0'; i++)  
    a[i] = b[i];
```

☐ 9☐ 10☒ 12☐ 11

Mantieni le tue informazioni salvando la risposta.

[Salva la mia risposta](#)[Torna alla pagina di ringraziamento](#)

10:01 S 📶 📷

5G 📶 77



forms.office.com

**✗ Non corretto** 0/1 Punti

12

Si consideri il seguente programma C,
quale tra le affermazioni elencate è vera?

```
int f(int *x){  
    int *y;  
    y = x;  
    (*y)++;  
    return *y;  
}  
  
int main(){  
    int a = 1;  
    int b = f(&a);  
}
```

☐ `a==2` e `b==1`☒ `a==1` e `b==2`☐ `\*a == \*b`☐ `&a == &b`☐ `a==2` e `b==2`**✗ Non corretto** 0/1 Punti

13

Qual è la lunghezza di `a` al termine del



10:01 S 📶 📷

5G 📶 77



forms.office.com

**✗ Non corretto** 0/1 Punti

11

Sia n un intero positivo maggiore di 1024 , qual è la complessità temporale dell'ultima riga del seguente frammento di codice C?

```
int i;  
float *a = malloc(n*sizeof(float));  
for (i = 0; i < n; i++)  
    a[i] = i;  
a = realloc(a, n/2);
```

- ☐ costante nel caso medio, lineare nel caso peggiore
- ☐ costante nel caso peggiore ✓
- ☒ lineare nel caso peggiore
- ☐ quadratico nel caso peggiore
- ☐ lineare nel caso medio

✗ Non corretto 0/1 Punti

09:19

5G 87



forms.office.com



12

(1 punto)

Sia a una lista Python di n interi e b una lista concatenata implementata in C avente gli stessi elementi di a . Siano $A(p)$ e $B(p)$ i costi dell'operazione di inserimento, rispettivamente in a e b , di un nuovo elemento in posizione p . Quale delle seguenti affermazioni è vera?

- ☐ $A(p) \leq B(p)$
- ☐ $B(p)$ è in $O(p)$
- ☐ $B(p) \leq A(p)$
- ☐ $A(p)$ è in $O(p)$

13

(1 punto)

Qual è la complessità temporale di $f(a)$?

```
def f(a):  
    d = {}  
    c = 0  
    for x in a:  
        if x in d:  
            c += 1  
        else:  
            d[x] = None  
    print(d)
```



09:19

5G 87



forms.office.com



11

(1 punto)

Si consideri il seguente frammento di codice: quale delle affermazioni è vera?

```
int a[3] = {1, 0, -1};
int *p;
char *c;

p = a;
c = a;

c++;
p++;
```

- ☐ `*p == *c`
- ☐ `c < p`
- ☐ `a[p] == 0`
- ☐ `*c < *p`

12

(1 punto)

Sia `a` una lista Python di `n` interi e `b` una lista concatenata implementata in C avente gli stessi elementi di `a`. Siano `A(p)` e `B(p)` i costi dell'operazione di inserimento, rispettivamente in `a` e `b`, di un nuovo elemento in posizione `p`. Quale delle seguenti affermazioni è vera?



09:19

5G 87



forms.office.com



10

(1 punto)

Sia n un intero positivo, assumendo che c sia una opportuna costante qual è lo spazio occupato dalla struttura d definita nel seguente frammento di codice?

```
a, d = [], {}  
  
for i in range(n):  
    a.append(i)  
    d[i] = a
```

- ☐ circa $n^{**}c$
- ☐ circa $c*n^{**}2$
- ☐ circa $c*n+(n^{**}2)/2$
- ☐ circa $c*(n^{**}2)/2$
- ☐ circa $c*n$

11

(1 punto)

Si consideri il seguente frammento di codice; quale delle affermazioni è vera?

```
int a[3] = {1, 0, -1};
```



09:19

5G 87



forms.office.com



9

(1 punto)

Quanto vale `c` al termine del seguente frammento di codice?

```
a = [3,2,5,4,5,6,7,6,7,5]
d = {}

c = 0
for x in a:
    if x in d:
        c += 1
    else:
        d[x] = None
```

- ☐ 4
- ☐ 1
- ☐ 3
- ☐ 10
- ☐ 7

10

(1 punto)



09:19

5G 87



forms.office.com



8

(1 punto)

Quanto vale `n` al termine del seguente frammento di codice?

```
char x[] = "1250";  
int n = sscanf(x, "%d\n", &n);
```

- ☐ 1
- ☐ il numero digitato dall'utente
- ☐ 1250
- ☐ 0

9

(1 punto)

Quanto vale `c` al termine del seguente frammento di codice?

```
a = [3,2,5,4,5,6,7,6,7,5]  
d = {}  
  
c = 0  
for x in a:  
    if x in d:  
        c += 1  
    else:
```



09:19

5G 87



forms.office.com



7

(1 punto)

Sia `d` un `dict` implementato in C come visto a lezione (le definizioni delle strutture sono riportate nel codice seguente). Assumendo che un `d_item` occupi `c` byte ed un puntatore `p` byte, quanto vale `sizeof(d)` ?

```
struct d_item {
    char *k;
    float v;
};
typedef struct d_item d_item;

struct nodo {
    d_item info;
    struct nodo *succ;
};
typedef struct nodo nodo;

struct dict {
    nodo **a;
    int m; // numero di liste (dimensione di a)
    int n; // numero di elementi nel dizionario
};
typedef struct dict dict;
```

- ☐ `O(n+m)`
- ☐ `O(1)`
- ☐ circa `p*d.m`
- ☐ circa `c*d.n`

8



09:19

5G 87



forms.office.com



...gli elementi e la più grande la più
piccolo in tempo quadratico nel caso peggiore

6

(1 punto)

Qual è la lunghezza di `a` al termine del seguente frammento di codice?

```
char a[] = "programmazione";  
char b[] = "algoritmo";  
int i;  
  
for(i=0; b[i] != '\0'; i++)  
    a[i] = b[i];
```

☐ 14☐ 9☐ 10☐ 15

7

(1 punto)

Sia `d` un `dict` implementato in C come visto a lezione (le definizioni delle strutture sono riportate nel codice seguente). Assumendo che un `d_item` occupi `c` byte ed un puntatore



09:19

5G 87



forms.office.com



5

Si consideri il seguente frammento di codice dove `a` è una lista di interi. Quale tra le affermazioni elencate è corretta?

(1 punto)

```
c = 0
n = len(a)

for c in range(n):
    i = 0
    while i < n-c:
        if a[-i] < a[-i-1]:
            a[-i], a[-i-1] = a[-i-1], a[-i]
            i += 1
```

- ☐ Ordina gli elementi di `a` dal più piccolo al più grande in tempo lineare nel caso migliore
- ☐ Ordina gli elementi di `a` dal più piccolo al più grande in tempo quadratico nel caso peggiore
- ☐ Il programma ha un comportamento indefinito e complessità temporale quadratica
- ☐ Ordina gli elementi di `a` dal più grande al più piccolo in tempo lineare nel caso migliore
- ☐ Ordina gli elementi di `a` dal più grande al più piccolo in tempo quadratico nel caso peggiore

6



09:19

5G 87



forms.office.com



4

Qual è il costo dell'algoritmo più efficiente che ordina k sequenze ordinate di lunghezza n/k ? (1 punto)

- ☐ $O(n \log k)$
- ☐ $O(n^k)$
- ☐ $O(n/k)$
- ☐ $O(n^2)$
- ☐ $O(kn)$
- ☐ $O(n)$

5

Si consideri il seguente frammento di codice dove `a` è una lista di interi. Quale tra le affermazioni elencate è corretta? (1 punto)

```
c = 0
n = len(a)

for c in range(n):
```



10:01 S 📶 📷

5G 📶 77



forms.office.com

**✗ Non corretto** 0/1 Punti

10

Quanto vale `tot`?

```
a = [1, 2, 3, 4]
b = [10, 20, 30, 40]
c = [a, b]
a = b[:]
c = [sum(x) for x in c]
tot = sum(c)
```

☐ 110☐ 210☐ 20☐ 100☒ 200**✗ Non corretto** 0/1 Punti

11

Sia `n` un intero positivo maggiore di



10:00 S 🌐 📷

5G 📶 77%



forms.office.com

**✗ Non corretto** 0/1 Punti

9

Sia t una tupla di n elementi e si consideri la seguente operazione di assegnazione in Python

$$x_0, x_1, \dots = t$$

dove a sinistra dell'uguale compaiono k variabili. In quale caso questo tipo di operazione è legale?

- ☒ solo se k è uguale a 1
- ☐ se k è uguale a n oppure a 1
- ☐ se k è maggiore o uguale a n
- ☐ se k è minore o uguale a n
- ☐ solo se k è uguale a n

**✗ Non corretto** 0/1 Punti

10:00 S 🌐 📷

5G 📶 77



forms.office.com

**✗ Non corretto** 0/1 Punti

8

Sia n un intero, qual è la complessità temporale del seguente frammento di codice?

```
int i, c = n/16;
int *a = malloc(sizeof(int)*c);

for (i = 0; i < n; i++){
    if (i == c){
        c = 2*c;
        a = malloc(c*sizeof(int));
    }
    a[i] = i;
}
```

- ☐ $O(n^2)$ caso peggiore, $O(n)$ caso migliore
- ☒ $O(n^2)$ caso peggiore, $O(n)$ caso medio
- ☐ $O(n^2)$ caso peggiore e caso migliore
- ☐ $O(n)$ caso peggiore e caso migliore ✓

✗ Non corretto 0/1 Punti

9

Sia t una tupla di n elementi e si



10:00 S 🌐 📷

5G 📶 77



forms.office.com



✗ Non corretto 0/1 Punti

7

Si consideri il seguente programma C,
quale tra le affermazioni elencate è vera?

```
char *f(char *x, int k){  
    return &(x[k]);  
}  
  
int main() {  
    char a[] = "programming";  
    char *x = f(a, 3);  
}
```

☒ `&a < &x`☐ `a < &x`☐ `\*a < \*x`☐ `&a < x`☐ `a < x` ✓

10:00 S 🌐 📷

5G 📶 77



forms.office.com



✓ Esatto 1/1 Punti

6

Siano `merge_sort()` e `bubble_sort()` due funzioni Python che implementano nel modo più ottimizzato possibile i due algoritmi omonimi. Sia inoltre `a` una lista di `n > 2c` numeri dove `c` un intero minore di `1024`, qual è la complessità temporale della seguente funzione?

```
def rsort(a):  
    merge_sort(a[:c])  
    bubble_sort(a)
```



- ☒ $O(n^{**2})$
- ☐ $O(n)$
- ☐ $O(n*n*\log n)$
- ☐ $O(n*\log n)$
- ☐ $O(n^{**2} * n*\log n)$
- ☐ $O(\min(n^{**2}, n*\log n))$



10:00 S 🌐 📷

5G 📶 77



forms.office.com



✓ Esatto 1/1 Punti

5

Sia a una lista di interi e siano x e y due suoi elementi, quale tra le seguenti affermazioni è sempre vera dopo l'esecuzione del seguente codice?

```
b = sorted(a, key=lambda x: ( 1 if x < 0 else 0, abs(x) ) )
```



- ☐ $y \leq x$ implica x precede y in b
- ☐ $x \leq y$ implica x precede y in b
- ☐ $x < 0$ e $y > 0$ implica x precede y in b
- ☒ $x < 0$ e $y > 0$ implica y precede x in b ✓

✓ Esatto 1/1 Punti

6

Siano `merge_sort()` e `bubble_sort()` due funzioni Python che implementano nel modo più ottimizzato possibile i due algoritmi omonimi. Sia inoltre `a` una



10:00 S 🌐 📷

5G 📶 77



forms.office.com

**✗ Non corretto** 0/1 Punti

4

Qual è il valore di `sum(b)` alla fine del seguente frammento di codice?

```
b = [0]
a = [b for i in range(10)]
for x in a:
    x[0] += 1
s = 0
for x in a:
    s += sum(x)
```

☐ 110☐ 90☐ 100☐ 10☒ 50☐ 0**✓ Esatto** 1/1 Punti

10:08

5G 65



forms.office.com

☐ tare lineare in n ☐ La funzione richiede tempo lineare e spazio supplementare costante

13

(1 punto)

Si consideri la seguente funzione, quale tra le affermazioni è vera?

```
char *f(char *a){  
    int n = strlen(a);  
    a = realloc(a, n/2);  
    a[n/2-1] = '\0';  
}
```

☐ La funzione richiede tempo costante e spazio supplementare costante☐ La funzione richiede tempo e spazio supplementare lineare in n ☐ La funzione richiede spazio lineare in n e tempo costante☒ La funzione richiede tempo lineare in n e spazio supplementare costante

Invia



10:08

5G 65



forms.office.com



Sia bubble sort che merge sort usano $O(n)$ spazio aggiuntivo.

12

(1 punto)

Si consideri la seguente funzione, quale tra le affermazioni è vera?

```
char *f(char *a){  
    int n = strlen(a);  
    char *b = malloc(n);  
    int i;  
    for (i = 0; i < 10; i++)  
        b[i] = a[i];  
    b[i] = '\0';  
    return b;  
}
```



La funzione richiede spazio lineare e tempo costante



La funzione richiede tempo costante e spazio supplementare costante



La funzione richiede tempo e spazio supplementare lineare in n



La funzione richiede tempo lineare e spazio supplementare costante

13



??

10:08

5G 65

☆

forms.office.com

↓ ↺

☐

$O(n*n)$

11

(1 punto)

Quale delle seguenti affermazioni sulla complessità spaziale aggiuntiva di **bubble sort** e **merge sort** è vera? (n è la lunghezza della lista di input)

☐

Sia bubble sort che merge sort usano $O(1)$ spazio aggiuntivo.

☐

Sia bubble sort che merge sort usano $O(n \log n)$ spazio aggiuntivo.

☒

Bubble sort usa $O(1)$ spazio aggiuntivo, mentre merge sort usa $O(n)$ spazio aggiuntivo.

☐

Bubble sort usa $O(n)$ spazio aggiuntivo, mentre merge sort usa $O(1)$ spazio aggiuntivo.

☐

Sia bubble sort che merge sort usano $O(n)$ spazio aggiuntivo.

12

(1 punto)

< > 🏠 ☆ 📅 ☰

10:08

5G 65



forms.office.com



- ☐ La funzione richiede tempo costante e spazio supplementare costante
- ☐ La funzione richiede tempo al più lineare in n e spazio supplementare lineare in n

9

(1 punto)

Siano a e b due dizionari Python tali che a sia molto più piccolo di b . Qual è la complessità temporale dell'algoritmo più efficiente per calcolare l'intersezione tra le chiavi di a e b ?

- ☐ $O(\text{len}(b))$
- ☐ $O(\text{len}(a))$
- ☒ $O(1)$
- ☐ $O(\text{len}(b) * \text{len}(a))$

10

(1 punto)

Quanta memoria richiede l'esecuzione della funzione `rsum()`?

```
def rsum(n):  
    if n > 0:
```



??

10:08

5G 65



forms.office.com



☐ $O(\text{len}(b) * \text{len}(a))$

10

(1 punto)

Quanta memoria richiede l'esecuzione della funzione `rsum()` ?

```
def rsum(n):  
    if n > 0:  
        return n+rsum(n-1)  
    else:  
        return 0
```

☐ $O(n \log n)$

☐ $O(1)$

☐ $O(n*1.5)$

☒ $O(n)$

☐ $O(n*n)$

11

(1 punto)

Quanta memoria richiede l'esecuzione della funzione `rsum()` ?



10:06

5G 65



forms.office.com



101

7

(1 punto)

Qual è l'output del seguente codice?

```
a = [1, 2, 1, 2, 3]
b = ['1', 'two', 'one', 'three', 2]
d = {}

for x, y in zip(a, b):
    d[x] = y

print(d)
```

- ☐ `{1: '1', 2: 'two', 3: 'three'}`
- ☐ `{1: '1', 2: 'two', 3: 'one'}`
- ☒ `{1: '1', 2: 'two', 1: 'one', 2: '2', 3: 'three'}`
- ☐ `{1: 'one', 2: '2', 3: 'three'}`
- ☐ `{1: 'one', 2: 'two', 3: 'three'}`
- ☐ `{1: 'one', 2: 'three', 3: 2}`

8



??

10:06

5G 65



forms.office.com



6

(1 punto)



Qual è l'output del seguente codice?

```
def mystery(x):  
    if x == 0:  
        return 0  
    return x % 2 + 10 * mystery(x // 2)  
  
mystery(4)
```

☐ 100

☒ 10

☐ 4

☐ 0

☐ 101

7

(1 punto)



Qual è l'output del seguente codice?

```
a = [1, 2, 1, 2, 3]
```



10:08

5G 65



forms.office.com



8

(1 punto)

Si consideri la seguente funzione, quale tra le affermazioni è vera?

```
char *f(char *a){  
    int n = strlen(a);  
    a = realloc(a, n*2);  
}
```



La funzione richiede spazio lineare in n e tempo costante



La funzione richiede tempo al più lineare in n e spazio supplementare costante



La funzione richiede tempo costante e spazio supplementare costante



La funzione richiede tempo al più lineare in n e spazio supplementare lineare in n

9

(1 punto)

Siano a e b due dizionari Python tali che a sia molto più piccolo di b . Qual è la complessità temporale dell'algoritmo più efficiente per calcolare l'intersezione tra le chiavi di a e b ?



10:06 5G 65



forms.office.com



Esaminante

4

(1 punto)

`L` è una lista contenente 4 stringhe di lunghezza `n`. Quale tra le affermazioni relative al seguente codice risulta essere falsa?

```
a = ''
for x in L[:-1]:
    for y in x:
        a += y

b = L[0] + L[1] + L[2]
```



La complessità temporale per la costruzione di ``b`` è lineare in ``n``



La lunghezza della stringa ``a`` è ``3n``



La complessità temporale per la costruzione di ``a`` è maggiore di quella per la costruzione di ``b``



La complessità temporale per la costruzione di ``a`` è lineare in ``n``



Le stringhe ``a`` e ``b`` sono uguali

5



??

10:06

5G 65

forms.office.com

5

(1 punto)

Qual è l'output del seguente codice?

```
total = 0
for i in range(3):
    for j in range(i, 3):
        total += j
print(total)
```

☐ 9

☐ 10

☐ 17

☐ 6

☒ 12

☐ 15

☐ 8

6

(1 punto)

12:59

76%



forms.office.com

**✗ Non corretto** 0/1 Punti

1

Sia N una variabile `int` inizializzata con un valore positivo, qual è il costo computazione, in termini di tempo di calcolo, del seguente frammento di codice C?

```
int size_a = N;
float *a = malloc(sizeof(float)*size_a);

for (int i = 0; i < 2*N; i++){
    if (i == size_a){
        float *t = a;
        a = malloc(sizeof(float)*(i+1));
        size_a = i+1;
        for(int j = 0; j < size_a; j++)
            a[j] = t[j];
        free(t);
    }
    a[i] = i;
}
```

- ☐ Sempre lineare in N
- ☐ Lineare in N nel caso medio, quadratico in N nel caso peggiore
- ☒ Sempre quadratico in N .
- ☐ Il tempo di calcolo non dipende da N
- ☐ Lineare in N nel caso peggiore, costante nel caso migliore.



12:59

76%



forms.office.com



✗ Non corretto 0/1 Punti

2

Qual è la complessità spaziale (quantità di memoria supplementare) della funzione `merge` utilizzata dall'algoritmo `merge-sort` ?

- ☐ lineare
- ☐ costante
- ☐ poli-logaritmica
- ☒ quadratica

✓ Esatto 1/1 Punti

3

Sia `a` una lista Python di lunghezza `n > 123`, dire quale delle seguenti affermazioni è falsa dopo aver eseguito il seguente frammento di codice.

```
b = a
b.append(n)
a = b + [ 2*n ]
```

- ☐ `min(a) == min(b)`



???

12:59

76%



forms.office.com



quadratica

✓ Esatto 1/1 Punti

3

Sia `a` una lista Python di lunghezza `n > 123`, dire quale delle seguenti affermazioni è falsa dopo aver eseguito il seguente frammento di codice.

```
b = a
b.append(n)
a = b + [ 2*n ]
```

☐ `min(a) == min(b)`

☐ `len(a) != len(b)`

☒ `a[-1] == b[-1]`



☐ `len(b) < max(a)`

☐ `max(a) >= max(b)`

✗ Non corretto 0/1 Punti

4

Sia `a` una lista contenente `bool`, `int` e `str`, cosa restituisce `a_func(a)`?



???

12:59

76%



forms.office.com



max(a) >= max(b)

✗ Non corretto 0/1 Punti

4

Sia `a` una lista contenente `bool`, `int` e `str`, cosa restituisce `a_func(a)`?

```
def a_func(a):  
    d = {}  
    for x in a:  
        if x in d:  
            d[x] += 1  
        else:  
            d[x] = 1  
    return max(a, key=lambda x: d[x])
```

- ☐ il massimo in `a`
- ☒ l'elemento che compare più frequentemente in `a` ✓
- ☐ la stringa più lunga in `a`
- ☐ il tipo che compare più frequentemente in `a`
- ☐ il numero di volte in cui compare l'elemento più frequente in `a`

✓ Esatto 1/1 Punti

5

Le funzioni `intersect0` e `intersect1` calcolano lo stesso valore



???

12:59

76%



forms.office.com



Esatto 1/1 Punt

5

Le funzioni `intersect0` e `intersect1` calcolano lo stesso valore, dire quale delle seguenti affermazioni è falsa.

```
def intersect0( a , b):
    for x in a:
        if x in b:
            return True
    return False

def intersect1( a, b ):
    A, B, c = {}, {}, 0
    for x in a:
        A[x] = None
    for x in b:
        B[x] = None

    for x in A.keys():
        if x in B:
            return True

    return False
```

- ☐ La memoria supplementare di **intersert0** è costante
- ☐ Nel caso peggiore, il tempo di calcolo di **intersert0** è quadratico
- ☐ Nel caso medio, il tempo di calcolo di **intersert1** è lineare
- ☐ La memoria supplementare di **intersert1** è lineare
- ☒ Nel caso medio, il tempo di calcolo di **intersert0** è lineare ✓



12:59

76%



forms.office.com



☐ se **b** è una qualsiasi stringa

☐ se **b** è una tupla vuota

✗ Non corretto 0/1 Punti

7

a è una lista Python contenente $\sqrt{n}$ interi ordinati in modo crescente seguiti da altri $\sqrt{n}$ interi non ordinati. Qual è il costo del miglior algoritmo che ordini la lista **a** ?

☐ $O(\sqrt{n} \log n)$



☒ $O(n^2)$

☐ $O(n \log n)$

☐ $O(\sqrt{n})$

☐ $O(n)$

✗ Non corretto 0/1 Punti

8

Sia **L** una lista concatenata di **n** interi implementata nel



12:59



forms.office.com



lineare

✓ **Esatto** 1/1 Punti

6

In quale caso l'esecuzione di `myfunc(b)` genera un errore?

```
def myfunc( a ):
    n = len(a)
    if n < 2:
        return a
    return myfunc(a[:n//2]+ a[n//2:])
```

- ☐ se il primo elemento di **b** è uno zero
- ☒ se **b** è una lista con almeno due elementi
- ☐ mai
- ☐ se **b** è una qualsiasi stringa
- ☐ se **b** è una tupla vuota

✘ Non corretto 0/1 Punti

7



???

13:00

76%



forms.office.com



✗ Non corretto 0/1 Punti

8

Sia L una lista concatenata di n interi implementata nel linguaggio C . Si consideri l'operazione **reset** che prende in input l'indirizzo del primo elemento di L ed un intero k e azzerà il valore dell'elemento in posizione k della lista. Qual è il costo computazionale in termini di tempo di calcolo della migliore implementazione di **reset**?

- ☒ lineare in k ✓
- ☐ quadratico in n
- ☐ lineare in n
- ☐ costante
- ☐ logaritmico in k

✗ Non corretto 0/1 Punti

9

Qual è la complessità temporale (tempo di calcolo) dell'algoritmo **merge** nel caso in cui le sequenze da fondere abbiano dimensione n e m ?



13:00

76%



forms.office.com



✗ Non corretto 0/1 Punti

10

Si consideri il seguente frammento di codice C, quale tra le seguenti affermazioni è vera?

```
int *f( int *k ){
    *k = 3*k;
    return k;
}

void main(){
    int m,n;

    /*
     * ...
     * operazioni in cui viene assegnato un valore a n
     * ...
     * */

    m = *f(&n);
}
```

☒ n ed m puntano allo stesso indirizzo

☐ n è 3 volte m

☐ m è un array

☐ n e m sono uguali



☐ m è 3 volte n

☐ *n è 3 volte *m



???

13:00

76%



forms.office.com



☒ logaritmico in k

✗ Non corretto 0/1 Punti

9

Qual è la complessità temporale (tempo di calcolo) dell'algoritmo merge nel caso in cui le sequenze da fondere abbiano dimensione n e m ?

☒ $O(m \log n)$

☐ $O(\min(n, m))$

☐ $O(n \log m)$

☐ $O(n * m)$

☐ $O(\max(n, m))$



✗ Non corretto 0/1 Punti

10

Si consideri il seguente frammento di codice C, quale tra le seguenti affermazioni è vera?

```
int *f( int *k ){
    *k = 3*k;
    return k;
}
```



11:14

0.73 KB/S

4G+

78

×

Esame 2024-01-24

forms.office.com

`a`

✓ Esatto

1/1 Punti

3

Sia h una funzione *hash* utilizzata per implementare un dizionario con n liste di trabocco. Supponiamo che vengano eseguite n operazioni di update con le seguenti coppie chiave-valore

$(k, v_1), (k, v_2), \dots, (k, v_n)$

Qual è la complessità temporale complessiva delle n operazioni nel caso peggiore?

☐

$O(1)$

☒

$O(n^2)$

✓

☐

$O(n^3)$

☐

$O(n)$

✓ Esatto

1/1 Punti

4

11:14

0.26 KB/S 4G+ 78

Esame 2024-01-24
forms.office.com

✗ Non corretto 0/1 Punti

5

Sia $\text{len}(a)=n$, qual è la complessità temporale di $f(a)$?

```
def f(a):  
    c = 0  
  
    if a == []:  
        return 0  
  
    for x in a:  
        c += len(a) + f(a[1:])  
  
    return c
```

☐ $O(n^2)$ ☐ $O(n^3)$ ☐ $O(2^n)$ ☐ $O(n)$ ☒ $O(1)$

✗ Non corretto 0/1 Punti

6

11:14

0.43 KB/S 4G+ 78

Esame 2024-01-24
forms.office.com

Punti: 4/10

✗ Non corretto 0/1 Punti

1

Sia n un intero, qual è la complessità temporale del seguente frammento di codice?

```
int i, c = n/16;  
int *a = malloc(sizeof(int)*c);  
  
for (i = 0; i < n; i++){  
    if (i == c){  
        c = 2*c;  
        a = malloc(c*sizeof(int));  
    }  
    a[i] = i;  
}
```

- ☐ $O(n^2)$ caso peggiore e caso migliore
- ☒ $O(n)$ caso peggiore e caso migliore ✓
- ☐ $O(n^2)$ caso peggiore, $O(n)$ caso medio
- ☐ $O(n^2)$ caso peggiore, $O(n)$ caso migliore

✗ Non corretto 0/1 Punti

11:14

0.13 KB/S 4G+ 78

Esame 2024-01-24
forms.office.com

✓ Esatto 1/1 Punti

4

Qual è il valore di `enigma( a )` ?

```
def enigma(a):  
    c = 0  
  
    for x in a:  
        try:  
            c += len(x) + enigma(a[1:])  
        except TypeError:  
            c = 0  
    return c  
  
a = [ [1,2, [3,4, [5,6]]], 0]
```

☐ 4☐ 3☐ 1☐ 2☒ 0

✗ Non corretto 0/1 Punti

11:13

0.36 KB/S 4G+ 78

Esame 2024-01-24
forms.office.com✗ **Non corretto** 0/1 Punti

6

In quale tra i casi elencati il seguente codice Python è corretto?

```
for x in a:  
    t += x
```

- ☐ Se `t` è inizializzato come tupla e `a` contiene solo tuple ✓
- ☐ Se `t` è inizializzato a `0`
- ☐ Se `t` è inizializzato come lista
- ☒ Se `t` è inizializzato come stringa vuota e `a` contiene delle stringhe
- ☐ Se `t` è inizializzato come lista e `a` è una lista che contiene anche altre liste

✓ **Esatto** 1/1 Punti

7

11:13

0.14 KB/S 4G+ 78

Esame 2024-01-24
forms.office.com☐ $O(n)$ caso peggiore e caso migliore

✓ Esatto 1/1 Punti

8

Quanto vale tot ?

```
a = [1,2,3,4]
b = [10, 20, 30, 40]
c = [a, b]
a = b[:]
c = [sum(x) for x in c]
tot = sum(c)
```

☐ 200☒ 110☐ 210☐ 100☐ 20

✗ Non corretto 0/1 Punti

0

11:14

0.73 KB/S 4G+ 78

Esame 2024-01-24
forms.office.com

migliore

✗ Non corretto 0/1 Punti

2

Quanto vale z al termine del seguente frammento di codice

```
int a[100] = {1, 0, -1};  
int *b = a;  
char *c = a;  
int z = a[10], w;  
  
c = c+1;  
b = b+1;  
  
if( b == c )  
    z = 0;  
else if (b < c)  
    z = -1;  
else if (b > c)  
    z = 1;  
else  
    z = a
```

☐ -1☒ 0☐ 1☐ 'a'**✓ Esatto** 1/1 Punti

11:13

0.36 KB/S

4G+

78

×

Esame 2024-01-24

forms.office.com

×

Non corretto

0/1 Punti

6

In quale tra i casi elencati il seguente codice Python è corretto?

```
for x in a:  
    t += x
```

☐

Se `t` è inizializzato come tupla e `a` contiene solo tuple

✓

☐

Se `t` è inizializzato a `0`

☐

Se `t` è inizializzato come lista

☒

Se `t` è inizializzato come stringa vuota e `a` contiene delle stringhe

☐

Se `t` è inizializzato come lista e `a` è una lista che contiene anche altre liste

✓

Esatto

1/1 Punti

7

11:14

0.73 KB/S

4G+

78

×

Esame 2024-01-24

forms.office.com

`a`

✓ Esatto

1/1 Punti

3

Sia h una funzione *hash* utilizzata per implementare un dizionario con n liste di trabocco. Supponiamo che vengano eseguite n operazioni di update con le seguenti coppie chiave-valore

$(k, v_1), (k, v_2), \dots, (k, v_n)$

Qual è la complessità temporale complessiva delle n operazioni nel caso peggiore?

☐

$O(1)$

☒

$O(n^2)$

✓

☐

$O(n^3)$

☐

$O(n)$

✓ Esatto

1/1 Punti

4

11:13

0.12 KB/S 4G+ 78

Esame 2024-01-24
forms.office.com

✗ Non corretto 0/1 Punti

10

Quanto vale `z` al termine del seguente frammento di codice?

```
int a[100] = {1, 0, -1};  
int *b = a;  
char *c = a;  
int z = a[10], w;  
  
c = c+1;  
b = b+1;  
  
if( b == c )  
    w = 0;  
else if (b < c)  
    w = -1;  
else  
    w = 1;  
else  
    w = a;  
  
printf("%d\n", z);
```

☐ -1☐ 1☐ 0☒ `a`

Mantieni le tue informazioni salvando la risposta.

Salva la mia risposta

11:13

0.36 KB/S 4G+ 78

Esame 2024-01-24
forms.office.com

Stare che contiene anche altre liste

✓ Esatto 1/1 Punti

7

Sia n un intero, qual è la complessità temporale del seguente frammento di codice?

```
int i, c = n/16;  
int *a = malloc(sizeof(int)*c);  
  
for (i = 0; i < n; i++){  
    if (i == c){  
        c = c+1;  
        a = realloc(a, c*sizeof(int));  
    }  
    a[i] = i;  
}
```

- ☐ $O(n^2)$ caso peggiore e caso migliore
- ☐ $O(n^2)$ caso peggiore, $O(n)$ caso medio
- ☒ $O(n^2)$ caso peggiore, $O(n)$ caso migliore ✓
- ☐ $O(n)$ caso peggiore e caso migliore

✓ Esatto 1/1 Punti

8

11:14

0.43 KB/S 4G+ 78

Esame 2024-01-24
forms.office.com

Punti: 4/10

✗ Non corretto 0/1 Punti

1

Sia n un intero, qual è la complessità temporale del seguente frammento di codice?

```
int i, c = n/16;  
int *a = malloc(sizeof(int)*c);  
  
for (i = 0; i < n; i++){  
    if (i == c){  
        c = 2*c;  
        a = malloc(c*sizeof(int));  
    }  
    a[i] = i;  
}
```

- ☐ $O(n^2)$ caso peggiore e caso migliore
- ☒ $O(n)$ caso peggiore e caso migliore ✓
- ☐ $O(n^2)$ caso peggiore, $O(n)$ caso medio
- ☐ $O(n^2)$ caso peggiore, $O(n)$ caso migliore

✗ Non corretto 0/1 Punti

11:14

0.73 KB/S 4G+ 78

Esame 2024-01-24
forms.office.com

migliore

✗ Non corretto 0/1 Punti

2

Quanto vale z al termine del seguente frammento di codice

```
int a[100] = {1, 0, -1};  
int *b = a;  
char *c = a;  
int z = a[10], w;  
  
c = c+1;  
b = b+1;  
  
if( b == c )  
    z = 0;  
else if (b < c)  
    z = -1;  
else if (b > c)  
    z = 1;  
else  
    z = a
```

☐ -1☒ 0☐ 1☐ `a`**✓ Esatto** 1/1 Punti

11:14

0.26 KB/S 4G+ 78

Esame 2024-01-24
forms.office.com**✗ Non corretto** 0/1 Punti

5

Sia $\text{len}(a)=n$, qual è la complessità temporale di $f(a)$?

```
def f(a):  
    c = 0  
  
    if a == []:  
        return 0  
  
    for x in a:  
        c += len(a) + f(a[1:])  
  
    return c
```

☐ $O(n^2)$ ☐ $O(n^3)$ ☐ $O(2^n)$ ☐ $O(n)$ ☒ $O(1)$ **✗ Non corretto** 0/1 Punti

6

11:13

0.14 KB/S 4G+ 78

Esame 2024-01-24
forms.office.com☐ $O(n)$ caso peggiore e caso migliore

✓ Esatto 1/1 Punti

8

Quanto vale tot ?

```
a = [1,2,3,4]
b = [10, 20, 30, 40]
c = [a, b]
a = b[:]
c = [sum(x) for x in c]
tot = sum(c)
```

☐ 200☒ 110☐ 210☐ 100☐ 20

✗ Non corretto 0/1 Punti

11:13

0.45 KB/S 4G+ 78

Esame 2024-01-24
forms.office.com

✗ Non corretto 0/1 Punti

9

Siano `merge_sort()` e `bubble_sort()` due funzioni Python che implementano nel modo più ottimizzato possibile i due algoritmi omonimi. Sia inoltre `a` una lista di numeri, qual è la complessità temporale della seguente funzione?

```
def rsort(a, n):  
    merge_sort(a)  
    bubble_sort(a)
```

- ☐ `O(n**2)`
- ☐ `O(n*log n)`
- ☐ `O(max (n**2, n*log n))`
- ☐ `O(n**2 + n*log n)`
- ☒ `O(n*n*log n)`
- ☐ `O(n)`



✗ Non corretto 0/1 Punti

10

11:13

0.45 KB/S 4G+ 78

Esame 2024-01-24
forms.office.com

✗ Non corretto 0/1 Punti

9

Siano `merge_sort()` e `bubble_sort()` due funzioni Python che implementano nel modo più ottimizzato possibile i due algoritmi omonimi. Sia inoltre `a` una lista di numeri, qual è la complessità temporale della seguente funzione?

```
def rsort(a, n):  
    merge_sort(a)  
    bubble_sort(a)
```

- ☐ `O(n**2)`
- ☐ `O(n*log n)`
- ☐ `O(max (n**2, n*log n))`
- ☐ `O(n**2 + n*log n)`
- ☒ `O(n*n*log n)`
- ☐ `O(n)`



✗ Non corretto 0/1 Punti

10

11:13

0.12 KB/S 4G+ 78

Esame 2024-01-24
forms.office.com**✗ Non corretto** 0/1 Punti

10

Quanto vale `z` al termine del seguente frammento di codice?

```
int a[100] = {1, 0, -1};  
int *b = a;  
char *c = a;  
int z = a[10], w;  
  
c = c+1;  
b = b+1;  
  
if( b == c )  
    w = 0;  
else if (b < c)  
    w = -1;  
else  
    w = 1;  
else  
    w = a;  
  
printf("%d\n", z);
```

☐ -1☐ 1☐ 0☒ `a`

Mantieni le tue informazioni salvando la risposta.

Salva la mia risposta

11:13

0.36 KB/S 4G+ 78



Esame 2024-01-24

forms.office.com



Storiche contiene anche altre liste

✓ Esatto 1/1 Punti

7

Sia n un intero, qual è la complessità temporale del seguente frammento di codice?

```
int i, c = n/16;
int *a = malloc(sizeof(int)*c);

for (i = 0; i < n; i++){
    if (i == c){
        c = c+1;
        a = realloc(a, c*sizeof(int));
    }
    a[i] = i;
}
```

- ☐ $O(n^2)$ caso peggiore e caso migliore
- ☐ $O(n^2)$ caso peggiore, $O(n)$ caso medio
- ☒ $O(n^2)$ caso peggiore, $O(n)$ caso migliore ✓
- ☐ $O(n)$ caso peggiore e caso migliore

✓ Esatto 1/1 Punti

8

10:50

4G+ 80%



forms.office.com/Pa



1

**✗ Non corretto** 0/1 Punti

6

Sia `d` un `dict` implementato in C come visto a lezione (le definizioni delle strutture sono riportate nel codice seguente). Assumendo che un `d_item` occupi `c` byte ed un puntatore `p` byte, quanto vale `sizeof(d)` ?

```
struct d_item {
    char *k;
    float v;
};
typedef struct d_item d_item;

struct nodo {
    d_item info;
    struct nodo *succ;
};
typedef struct nodo nodo;

struct dict {
    nodo **a;
    int m; // numero di liste (dimensione di a)
    int n; // numero di elementi nel dizionario
};
typedef struct dict dict;
```

☐ circa ``p*d.m``☒ circa ``c*d.n``☐ ``O(n+m)``☐ ``O(1)``

10:51

4G+ 80%

forms.office.com/Pa

+

1

⋮

✕ Non corretto 0/1 Punti

10

Quanto vale `n` al termine del seguente frammento di codice?

```
char x[] = "1250";  
int n = sscanf(x, "%d\n", &n);
```

☐ 1

✓

☒ 1250

☐ il numero digitato dall'utente

☐ 0

Mantieni le tue informazioni salvando la risposta.

Salva la mia risposta

[Torna alla pagina di ringraziamento](#) Microsoft 365

10:50

4G+ 80%



forms.office.com/Pa



1



✗ Non corretto 0/1 Punti

4

Qual è la lunghezza di `a` al termine del seguente frammento di codice?

```
char a[] = "programmazione";  
char b[] = "algoritmo";  
int i;  
  
for(i=0; b[i] != '\0'; i++)  
    a[i] = b[i];
```

☒ 10

☐ 9

☐ 14

☐ 15

✗ Non corretto 0/1 Punti

5

Si consideri il seguente frammento di codice: quale delle affermazioni è vera?

```
int a[3] = {1, 0, -1};
```

10:50

4G+ 80%



forms.office.com/Pa



1



Punti: 1/10

✗ Non corretto 0/1 Punti

1

Qual è il costo dell'algoritmo più efficiente che ordini k sequenze ordinate ciascuna di dimensione n/k ?

☐ $O(n^{**k})$ ☐ $O(n)$ ☐ $O(k*n)$ ☐ $O(n*\log k)$ ☒ $O(n/k)$ ☐ $O(n^{**2})$ **✓ Esatto** 1/1 Punti

2

10:50

4G+ 80%



forms.office.com/Pa



1



✗ Non corretto 0/1 Punti

3

Quanto vale `c` al termine del seguente frammento di codice?

```
a = [3,2,5,4,5,6,7,6,7,5]
d = {}

c = 0
for x in a:
    if x in d:
        c += 1
    else:
        d[x] = None
```

☐ 3☐ 4☒ 1☐ 10☐ 7

✗ Non corretto 0/1 Punti

4

10:50

4G+ 80%



forms.office.com/Pa



1

☐ $O(n^{**2})$

✓ Esatto 1/1 Punti

2

Sia `a` una lista Python di `n` interi e `b` una lista concatenata implementata in C avente gli stessi elementi di `a`. Siano $A(p)$ e $B(p)$ i costi dell'operazione di inserimento, rispettivamente in `a` e `b`, di un nuovo elemento in posizione `p`. Quale delle seguenti affermazioni è vera?

☐ $B(p) \leq A(p)$ ☒ $B(p)$ è in $O(p)$ ✓☐ $A(p) \leq B(p)$ ☐ $A(p)$ è in $O(p)$

✗ Non corretto 0/1 Punti

3

Quanto vale `c` al termine del seguente frammento di codice?

```
a = [3, 2, 5, 4, 5, 6, 7, 6, 7, 5]
d = {}
```

10:50

4G+ 80%



forms.office.com/Pa



1



✗ Non corretto 0/1 Punti

7

Si consideri il seguente frammento di codice dove `a` è una stringa di interi, quale tra le affermazioni elencate è corretta?

```
c = 0
n = len(a)

for c in range(n):
    i = 0
    while i < n-c:
        if a[-i] < a[-i-1]:
            a[-i], a[-i-1] = a[-i-1], a[-i]
            i += 1
```

- ☒ Ordina gli elementi di `a` dal più piccolo al più grande in tempo lineare nel caso migliore
- ☐ Ordina gli elementi di `a` dal più grande al più piccolo in tempo quadratico nel caso peggiore
- ☐ Ordina gli elementi di `a` dal più piccolo al più grande in tempo quadratico nel caso peggiore
- ☐ Il programma ha un comportamento indefinito e complessità temporale quadratica ✓
- ☐ Ordina gli elementi di `a` dal più grande al più piccolo in tempo lineare nel caso migliore

10:51

4G+ 80%



forms.office.com/Pa



1

**✗ Non corretto** 0/1 Punti

9

Sia n un intero positivo, assumendo che c sia una opportuna costante qual è lo spazio occupato dalla struttura d definita nel seguente frammento di codice?

```
a, d = [], {}  
  
for i in range(n):  
    a.append(i)  
    d[i] = a
```

- ☐ circa $c \cdot n$
- ☐ circa $n^{**}c$
- ☒ circa $c \cdot n^{**}2$
- ☐ circa $c \cdot n + (n^{**}2)/2$
- ☐ circa $c \cdot (n^{**}2)/2$

**✗ Non corretto** 0/1 Punti

10





10:51

4G+ 80%



forms.office.com/Pa



1



✗ Non corretto 0/1 Punti

10

Quanto vale `n` al termine del seguente frammento di codice?

```
char x[] = "1250";  
int n = sscanf(x, "%d\n", &n);
```



1



1250



il numero digitato dall'utente



0

Mantieni le tue informazioni salvando la risposta.

Salva la mia risposta

[Torna alla pagina di ringraziamento](#)



Microsoft 365

10:50

4G+ 80%



forms.office.com/Pa



1

**✗ Non corretto** 0/1 Punti

6

Sia `d` un `dict` implementato in C come visto a lezione (le definizioni delle strutture sono riportate nel codice seguente). Assumendo che un `d_item` occupi `c` byte ed un puntatore `p` byte, quanto vale `sizeof(d)` ?

```
struct d_item {
    char *k;
    float v;
};
typedef struct d_item d_item;

struct nodo {
    d_item info;
    struct nodo *succ;
};
typedef struct nodo nodo;

struct dict {
    nodo **a;
    int m; // numero di liste (dimensione di a)
    int n; // numero di elementi nel dizionario
};
typedef struct dict dict;
```

☐ circa ``p*d.m``☒ circa ``c*d.n``☐ ``O(n+m)``☐ ``O(1)``

10:51

4G+ 80%



forms.office.com/Pa



1

**✗ Non corretto** 0/1 Punti

9

Sia n un intero positivo, assumendo che c sia una opportuna costante qual è lo spazio occupato dalla struttura d definita nel seguente frammento di codice?

```
a, d = [], {}  
  
for i in range(n):  
    a.append(i)  
    d[i] = a
```

- ☐ circa $c \cdot n$
- ☐ circa $n \cdot c$
- ☒ circa $c \cdot n^2$
- ☐ circa $c \cdot n + (n^2)/2$
- ☐ circa $c \cdot (n^2)/2$

**✗ Non corretto** 0/1 Punti

10



10:50

4G+ 80%



forms.office.com/Pa



1

**✗ Non corretto** 0/1 Punti

7

Si consideri il seguente frammento di codice dove `a` è una stringa di interi, quale tra le affermazioni elencate è corretta?

```
c = 0
n = len(a)

for c in range(n):
    i = 0
    while i < n-c:
        if a[-1] < a[-i-1]:
            a[-1], a[-i-1] = a[-i-1], a[-1]
            i += 1
```

- ☒ Ordina gli elementi di `a` dal più piccolo al più grande in tempo lineare nel caso migliore
- ☐ Ordina gli elementi di `a` dal più grande al più piccolo in tempo quadratico nel caso peggiore
- ☐ Ordina gli elementi di `a` dal più piccolo al più grande in tempo quadratico nel caso peggiore
- ☐ Il programma ha un comportamento indefinito e complessità temporale quadratica ✓
- ☐ Ordina gli elementi di `a` dal più grande al più piccolo in tempo lineare nel caso migliore

10:50

4G+ 80%



forms.office.com/Pa



1

☐ $O(n^{**2})$ ✓ **Esatto** 1/1 Punti

2

Sia `a` una lista Python di `n` interi e `b` una lista concatenata implementata in C avente gli stessi elementi di `a`. Siano $A(p)$ e $B(p)$ i costi dell'operazione di inserimento, rispettivamente in `a` e `b`, di un nuovo elemento in posizione `p`. Quale delle seguenti affermazioni è vera?

☐ $B(p) \leq A(p)$ ☒ $B(p)$ è in $O(p)$ ✓☐ $A(p) \leq B(p)$ ☐ $A(p)$ è in $O(p)$ ✗ **Non corretto** 0/1 Punti

3

Quanto vale `c` al termine del seguente frammento di codice?

```
a = [3, 2, 5, 4, 5, 6, 7, 6, 7, 5]
d = {}
```


10:50

4G+ 80%



forms.office.com/Pa



1



✗ Non corretto 0/1 Punti

3

Quanto vale `c` al termine del seguente frammento di codice?

```
a = [3,2,5,4,5,6,7,6,7,5]
d = {}

c = 0
for x in a:
    if x in d:
        c += 1
    else:
        d[x] = None
```

☐ 3☐ 4☒ 1☐ 10☐ 7

✗ Non corretto 0/1 Punti

4

10:51

4G+ 80%

forms.office.com/Pa

1

✖ Non corretto 0/1 Punti

10

Quanto vale `n` al termine del seguente frammento di codice?

```
char x[] = "1250";  
int n = sscanf(x, "%d\n", &n);
```

☐ 1

☒ 1250

☐ il numero digitato dall'utente

☐ 0

Mantieni le tue informazioni salvando la risposta.

Salva la mia risposta

[Torna alla pagina di ringraziamento](#)

 Microsoft 365

10:50

4G+ 80%



forms.office.com/Pa



1



✗ Non corretto 0/1 Punti

5

Si consideri il seguente frammento di codice: quale delle affermazioni è vera?

```
int a[3] = {1, 0, -1};  
int *p;  
char *c;  
  
p = a;  
c = a;  
  
c++;  
p++;
```

- ☐ `\*c < \*p`
- ☒ `\*p == \*c`
- ☐ `c < p`
- ☐ `a[p] == 0`



✗ Non corretto 0/1 Punti

6

10:50

4G+ 80%



forms.office.com/Pa



1



Punti: 1/10

✗ Non corretto 0/1 Punti

1

Qual è il costo dell'algoritmo più efficiente che ordini k sequenze ordinate ciascuna di dimensione n/k ?

☐ $O(n^{**k})$ ☐ $O(n)$ ☐ $O(k*n)$ ☐ $O(n*\log k)$ ☒ $O(n/k)$ ☐ $O(n^{**2})$ **✓ Esatto** 1/1 Punti

2

10:51

4G+ 80%



forms.office.com/Pa



1

**✗ Non corretto** 0/1 Punti

8

Qual è la complessità temporale di `f(a)` ?

```
def f(a):  
    d = {}  
    c = 0  
    for x in a:  
        if x in d:  
            c += 1  
        else:  
            d[x] = None  
            print(d)  
    print(d)  
    return c
```

- ☐ Lineare nel caso medio
- ☐ Quadratica nel caso medio
- ☒ Quadratica sempre
- ☐ Quadratica nel caso peggiore
- ☐ Lineare nel caso peggiore

**✗ Non corretto** 0/1 Punti

10:50

4G+ 80%



forms.office.com/Pa



1

**✗ Non corretto** 0/1 Punti

4

Qual è la lunghezza di `a` al termine del seguente frammento di codice?

```
char a[] = "programmazione";  
char b[] = "algoritmo";  
int i;  
  
for(i=0; b[i] != '\0'; i++)  
    a[i] = b[i];
```

☒ 10☐ 9☐ 14☐ 15**✗ Non corretto** 0/1 Punti

5

Si consideri il seguente frammento di codice: quale delle affermazioni è vera?

```
int a[3] = {1, 0, -1};
```



✗ Non corretto 0/1 Punti

6

Sia `d` un `dict` implementato in C come visto a lezione (le definizioni delle strutture sono riportate nel codice seguente). Assumendo che un `d_item` occupi `c` byte ed un puntatore `p` byte, quanto vale `sizeof(d)` ?

```
struct d_item {
    char *k;
    float v;
};
typedef struct d_item d_item;

struct nodo {
    d_item info;
    struct nodo *succ;
};
typedef struct nodo nodo;

struct dict {
    nodo **a;
    int m; // numero di liste (dimensione di a)
    int n; // numero di elementi nel dizionario
};
typedef struct dict dict;
```

☐ circa `p*d.m`

☒ circa `c*d.n`

☐ `O(n+m)`

☐ `O(1)`



10:50

4G+ 80%



forms.office.com/Pa



1



✗ Non corretto 0/1 Punti

5

Si consideri il seguente frammento di codice: quale delle affermazioni è vera?

```
int a[3] = {1, 0, -1};  
int *p;  
char *c;  
  
p = a;  
c = a;  
  
c++;  
p++;
```

- ☐ `\*c < \*p`
- ☒ `\*p == \*c`
- ☐ `c < p`
- ☐ `a[p] == 0`



✗ Non corretto 0/1 Punti

6



10:50

4G+ 80%



forms.office.com/Pa



1



✗ Non corretto 0/1 Punti

3

Quanto vale `c` al termine del seguente frammento di codice?

```
a = [3,2,5,4,5,6,7,6,7,5]
d = {}

c = 0
for x in a:
    if x in d:
        c += 1
    else:
        d[x] = None
```

☐ 3☐ 4☒ 1☐ 10☐ 7

✗ Non corretto 0/1 Punti

4



10:50

4G+ 80%



forms.office.com/Pa



1



✗ Non corretto 0/1 Punti

4

Qual è la lunghezza di `a` al termine del seguente frammento di codice?

```
char a[] = "programmazione";  
char b[] = "algoritmo";  
int i;  
  
for(i=0; b[i] != '\0'; i++)  
    a[i] = b[i];
```

☒ 10

☐ 9

☐ 14

☐ 15



✗ Non corretto 0/1 Punti

5

Si consideri il seguente frammento di codice: quale delle affermazioni è vera?

```
int a[3] = {1, 0, -1};
```

10:50

4G+ 80%



forms.office.com/Pa



1

☐ $O(n^{**2})$

✓ Esatto 1/1 Punti

2

Sia `a` una lista Python di `n` interi e `b` una lista concatenata implementata in C avente gli stessi elementi di `a`. Siano $A(p)$ e $B(p)$ il costi dell'operazione di inserimento, rispettivamente in `a` e `b`, di un nuovo elemento in posizione `p`. Quale delle seguenti affermazioni è vera?

☐ $B(p) \leq A(p)$ ☒ $B(p)$ è in $O(p)$ ✓☐ $A(p) \leq B(p)$ ☐ $A(p)$ è in $O(p)$

✗ Non corretto 0/1 Punti

3

Quanto vale `c` al termine del seguente frammento di codice?

```
a = [3, 2, 5, 4, 5, 6, 7, 6, 7, 5]
d = {}
```

10:50

4G+ 80%



forms.office.com/Pa



1



Punti: 1/10

✗ Non corretto 0/1 Punti

1

Qual è il costo dell'algoritmo più efficiente che ordini k sequenze ordinate ciascuna di dimensione n/k ?

☐ $O(n^{**k})$ ☐ $O(n)$ ☐ $O(k*n)$ ☐ $O(n*\log k)$ ☒ $O(n/k)$ ☐ $O(n^{**2})$ **✓ Esatto** 1/1 Punti

2

10:51

4G+ 80%



forms.office.com/Pa



1



✗ Non corretto 0/1 Punti

8

Qual è la complessità temporale di `f(a)` ?

```
def f(a):  
    d = {}  
    c = 0  
    for x in a:  
        if x in d:  
            c += 1  
        else:  
            d[x] = None  
            print(d)  
    print(d)  
    return c
```

- ☐ Lineare nel caso medio
- ☐ Quadratica nel caso medio
- ☒ Quadratica sempre
- ☐ Quadratica nel caso peggiore
- ☐ Lineare nel caso peggiore



✗ Non corretto 0/1 Punti

10:51

4G+ 80%



forms.office.com/Pa



1

**✗ Non corretto** 0/1 Punti

9

Sia n un intero positivo, assumendo che c sia una opportuna costante qual è lo spazio occupato dalla struttura d definita nel seguente frammento di codice?

```
a, d = [], {}  
  
for i in range(n):  
    a.append(i)  
    d[i] = a
```

- ☐ circa $c \cdot n$
- ☐ circa $n^{**}c$
- ☒ circa $c \cdot n^{**}2$
- ☐ circa $c \cdot n + (n^{**}2)/2$
- ☐ circa $c \cdot (n^{**}2)/2$

**✗ Non corretto** 0/1 Punti

10



10:50

4G+ 80%



forms.office.com/Pa



1

**✗ Non corretto** 0/1 Punti

7

Si consideri il seguente frammento di codice dove `a` è una stringa di interi, quale tra le affermazioni elencate è corretta?

```
c = 0
n = len(a)

for c in range(n):
    i = 0
    while i < n-c:
        if a[-1] < a[-i-1]:
            a[-1], a[-i-1] = a[-i-1], a[-1]
            i += 1
```

- ☒ Ordina gli elementi di `a` dal più piccolo al più grande in tempo lineare nel caso migliore
- ☐ Ordina gli elementi di `a` dal più grande al più piccolo in tempo quadratico nel caso peggiore
- ☐ Ordina gli elementi di `a` dal più piccolo al più grande in tempo quadratico nel caso peggiore
- ☐ Il programma ha un comportamento indefinito e complessità temporale quadratica ✓
- ☐ Ordina gli elementi di `a` dal più grande al più piccolo in tempo lineare nel caso migliore

10:51

4G+ 80%



forms.office.com/Pa



1

**✗ Non corretto** 0/1 Punti

8

Qual è la complessità temporale di `f(a)` ?

```
def f(a):  
    d = {}  
    c = 0  
    for x in a:  
        if x in d:  
            c += 1  
        else:  
            d[x] = None  
            print(d)  
    print(d)  
    return c
```

- ☐ Lineare nel caso medio
- ☐ Quadratica nel caso medio
- ☒ Quadratica sempre
- ☐ Quadratica nel caso peggiore
- ☐ Lineare nel caso peggiore

✗ Non corretto 0/1 Punti

10:50

4G+ 80%



forms.office.com/Pa



1



✗ Non corretto 0/1 Punti

5

Si consideri il seguente frammento di codice: quale delle affermazioni è vera?

```
int a[3] = {1, 0, -1};  
int *p;  
char *c;  
  
p = a;  
c = a;  
  
c++;  
p++;
```

- ☐ `\*c < \*p`
- ☒ `\*p == \*c`
- ☐ `c < p`
- ☐ `a[p] == 0`



✗ Non corretto 0/1 Punti

6



??

10:17 4G 68

☐ T è lineare in n , M è costante

☐ T è ordine di $n \log n$, M è costante

✗ Non corretto 0/1 Punti

7

Qual è il tempo di calcolo di $f(n)$ nel caso peggiore?

```
int *f(int n){
    int i, *a = NULL;
    for(i = 0; i < n; i++){
        a = realloc(a, (i+1)*sizeof(int));
        a[i] = i;
    }
    return a;
}
```

☐ costante

☐ ordine di $n \log n$

☐ quadratico in n

☒ lineare in n

✓ Esatto 1/1 Punti

8



forms.office.com





10:50

4G 82

AA

forms.office.com

☐ `c == 0`

✓ Esatto 1/1 Punti

9

Si assuma che **a** e **b** siano due dizionari, qual è la complessità temporale nel caso medio dell'operazione che restituisce la lista delle chiavi in comune tra **a** e **b**?

☐ $O(\text{len}(a) * \text{len}(b))$ ☐ $O(\text{len}(a) // \text{len}(b))$ ☒ $O(\min(\text{len}(a), \text{len}(b)))$ ✓☐ $O(\text{len}(b) // \text{len}(a))$ ☐ $O(\max(\text{len}(a), \text{len}(b)))$

✓ Esatto 1/1 Punti

10





10:50

4G 82

AA

forms.office.com

✓ **Esatto** 1/1 Punti

10

La lista **a** contiene **n** tuple costituite da due elementi ciascuna. Il primo è una etichetta che può valere **0** oppure **1**. Si vuole ordinare **a** in modo che le tuple con etichetta **0** precedano quelle con etichetta **1**. Qual è il costo temporale del miglior algoritmo che risolva il problema?

☐ $O(n^2)$ ☒ $O(n)$ ✓☐ $O(n \log n)$ ☐ $O(1)$ ☐ $O(\log n)$

Mantieni le tue informazioni salvando la risposta.

Salva la mia risposta



10:50

4G 82

AA

forms.office.com

☒ Qualcosa di indefinito

✓ Esatto 1/1 Punti

8

Sia n un intero positivo maggiore di 1876. si consideri il valore di c al termine del seguente frammento di codice, quale tra le affermazioni elencate (espresse in codice C) è vera?

```
int c = n;

for (int i = 0; i < n; i++){
    c -= 1;
    i += 1;
}
```

☐ $c == (n/2 - 1)$ ☐ $c == (n/2 + 1)$ ☐ $c == n$ ☒ $c == n/2$ ☐ $c == 0$

✓ Esatto 1/1 Punti





10:50

4G 82

AA

forms.office.com

✓ **Esatto** 1/1 Punti

7

Cosa succede eseguendo il seguente programma?

```
#include <stdio.h>
#include <string.h>

struct str {
    char *a;
    int n;
};
typedef struct str str;

str new_str(){
    str s;
    char x[] = "malloc";
    s.a = x;
    s.n = strlen(x);
    return s;
}

int main(){
    str b = new_str();
    printf("%s\n", b.a);
    return 6;
}
```

- ☐ Errore in compilazione
- ☐ Stampa la stringa malloc
- ☐ Stampa l'intero 6
- ☐ Un ciclo infinito
- ☒ Qualcosa di indefinito





10:50

4G 82

AA

forms.office.com



✓ Esatto 1/1 Punti

6

Sia n un intero positivo maggiore di 1876, qual è il valore di c al termine del seguente frammento di codice?

```
c = n
for i in range(n):
    c -= 1
    i += 1
```



- ☐ $n//2$
- ☒ 0
- ☐ n
- ☐ $n//2-1$
- ☐ $n//2+1$



✓ Esatto 1/1 Punti

7

Cosa succede eseguendo il seguente programma?

```
#include <stdio.h>
```





10:50

4G 82

AA

forms.office.com

☒ $O(n)$ ☐ $O(n^{**2})$ ☐ $O(n \log n)$ ✓ **Esatto** 1/1 Punti

5

Sia a una lista di n dizionari ciascuno contenente c elementi a valori `None`. Si consideri il seguente codice:

```
d = {}  
  
for i in range(len(a)):  
    d = unione(d, a[i])
```



La funzione `unione` ritorna un dizionario che contiene l'unione delle chiavi dei dizionari in input.

Assumendo che c sia una costante, qual è la complessità temporale del precedente codice?

☐ $O(n^{**3})$ ☐ $O(n \log n)$ ☐ $O(2^{**n})$ ☐ $O(n)$ 



10:50

4G 82

AA

forms.office.com



- ☐ Mai
- ☐ Solo se **a** è ordinata
- ☒ Se **a** è non ordinata



✓ **Esatto** 1/1 Punti

4

Si consideri una lista **a** contenente **n** interi ordinati dal più piccolo al più grande. Si definisca **b** come **b = [e] + a**. Qual è il costo temporale del bubble-sort sulla lista **b**?

- ☐ $O(1)$
- ☐ $O((n+1)^{**2})$
- ☒ $O(n)$
- ☐ $O(n^{**2})$
- ☐ $O(n \log n)$





10:50

4G 82

AA

forms.office.com



- ☒ La compilazione viene interrotta riportando un errore ✓
- ☐ La compilazione termina con successo riportando un warning

✓ Esatto 1/1 Punti

3

Sia **a** una lista di interi, in quale casi tra quelli elencati la seguente espressione viene valutata **True** ?

$$2 * \text{sorted}(a) \leq a$$

- ☐ Solo se **a** è vuota
- ☐ Sempre
- ☐ Mai
- ☐ Solo se **a** è ordinata
- ☒ Se **a** è non ordinata ✓





10:49

4G 82

AA

forms.office.com



☐ indefinito

☐ 11

✓ **Esatto** 1/1 Punti

2

Cosa succede compilando il seguente codice?

```
#include <stdio.h>

int main(){
    printf("%f\n", f(7));
}

float f(n){
    return 7/2.0;
}
```



- ☐ La compilazione termina con successo ma il codice contiene un errore semantico
- ☐ La compilazione termina con successo
- ☐ La compilazione non termina, deve essere interrotta manualmente
- ☒ La compilazione viene interrotta riportando un errore ✓
- ☐ La compilazione termina con successo riportando



??

10:49

5G

☐ T è ordine di $n \log n$, M è costante

✗ Non corretto 0/1 Punti

9

Quant'è la memoria supplementare utilizzata da $f(n)$ nel caso peggiore?

```
int *f(int n){
    int i, *a = NULL;
    for(i = 0; i < n; i++){
        a = realloc(a, (i+1)*sizeof(int));
        a[i] = i;
    }
    return a;
}
```

☐ ordine di $n \log n$

☒ quadratico in n

☐ lineare in n

☐ costante



✗ Non corretto 0/1 Punti

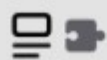


forms.office.com



???

10:28



forms.office.com



✗ Non corretto 0/1 Punti

9

Si consideri il seguente frammento di codice C, quale tra le seguenti affermazioni è vera?

```
int *f( int *k ){
    *k = 3*k;
    return k;
}

void main(){
    int m,n;

    /*
     * ...
     * operazioni in cui viene assegnato un valore a n
     * ...
     * */

    m = *f(&n);
}
```

- ☐ *n è 3 volte *m
- ☐ n è 3 volte m
- ☐ m è un array
- ☒ m è 3 volte n
- ☐ n e m sono uguali



10:53

4G 89

4

(1 punto)

Qual è il valore di `enigma( a )` ?

```
def enigma(a):  
    c = 0  
  
    for x in a:  
        try:  
            c += len(x) + enigma(a[1:])  
        except TypeError:  
            c = 0  
    return c  
  
a = [ [1,2, [3,4, [5,6]]], 0]
```

☐ 4☐ 0☐ 2☐ 3☐ 1

5

AA

forms.office.com



10:49  4G  82AA  forms.office.com

Nelle possibili soluzioni, tra *backtik* è indicato il codice sorgente oppure funzioni matematiche: esempio ``a += 1``

Punti: 10/10

✓ **Esatto** 1/1 Punti

1

Si consideri il seguente codice

```
int main(){  
    char a[100] = {11};  
    char *p = &a[50];  
}
```

Qual è il valore di `p`?

- ☐ 0
- ☐ 50
- ☒ a+50
- ☐ indefinito
- ☐ 11



10:53

4G 89

4

(1 punto)

Qual è il valore di `enigma( a )` ?

```
def enigma(a):  
    c = 0  
  
    for x in a:  
        try:  
            c += len(x) + enigma(a[1:])  
        except TypeError:  
            c = 0  
    return c  
  
a = [ [1,2, [3,4, [5,6]]], 0]
```

- ☐ 4
- ☐ 0
- ☐ 2
- ☐ 3
- ☐ 1

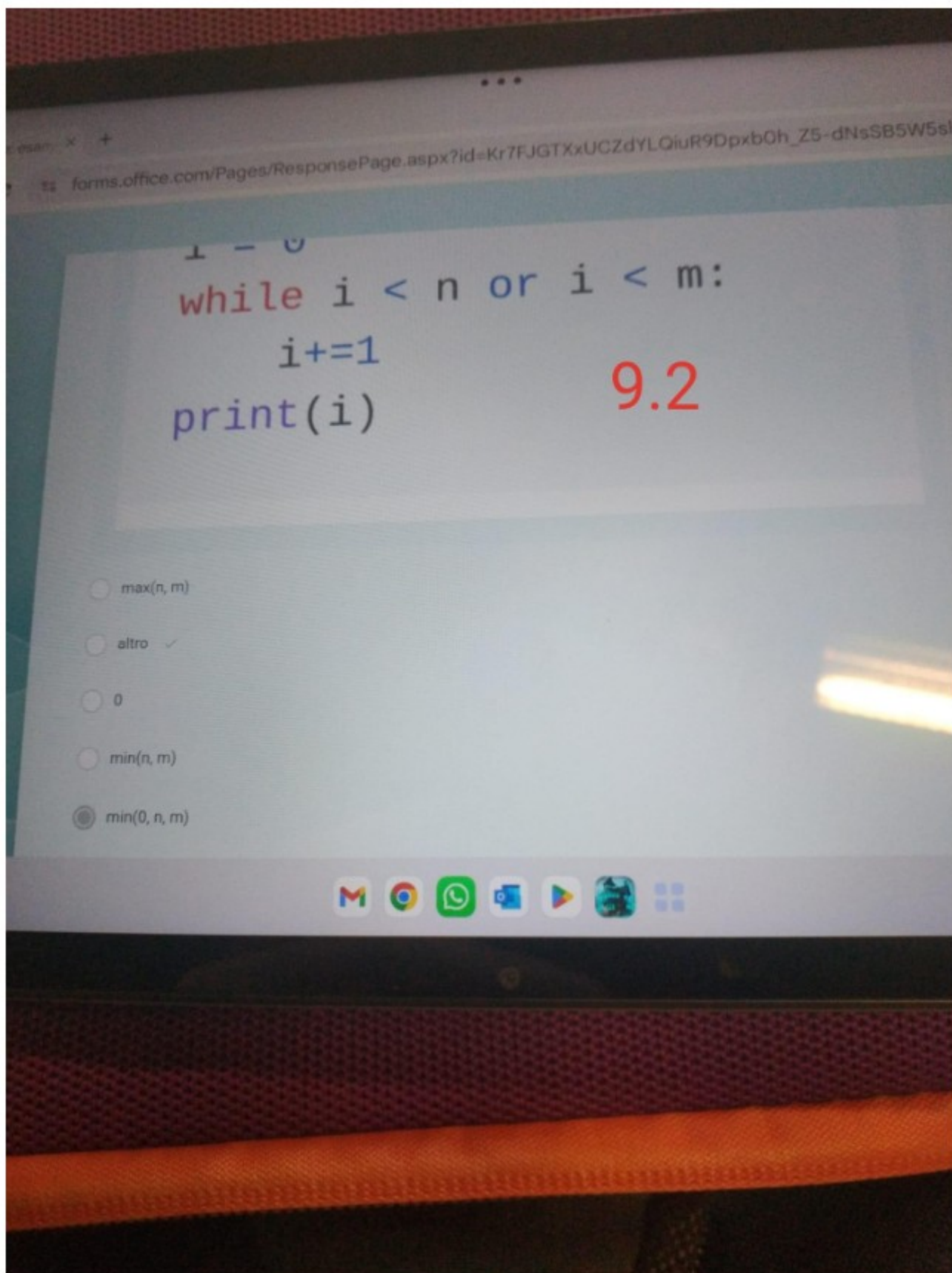
5

AA

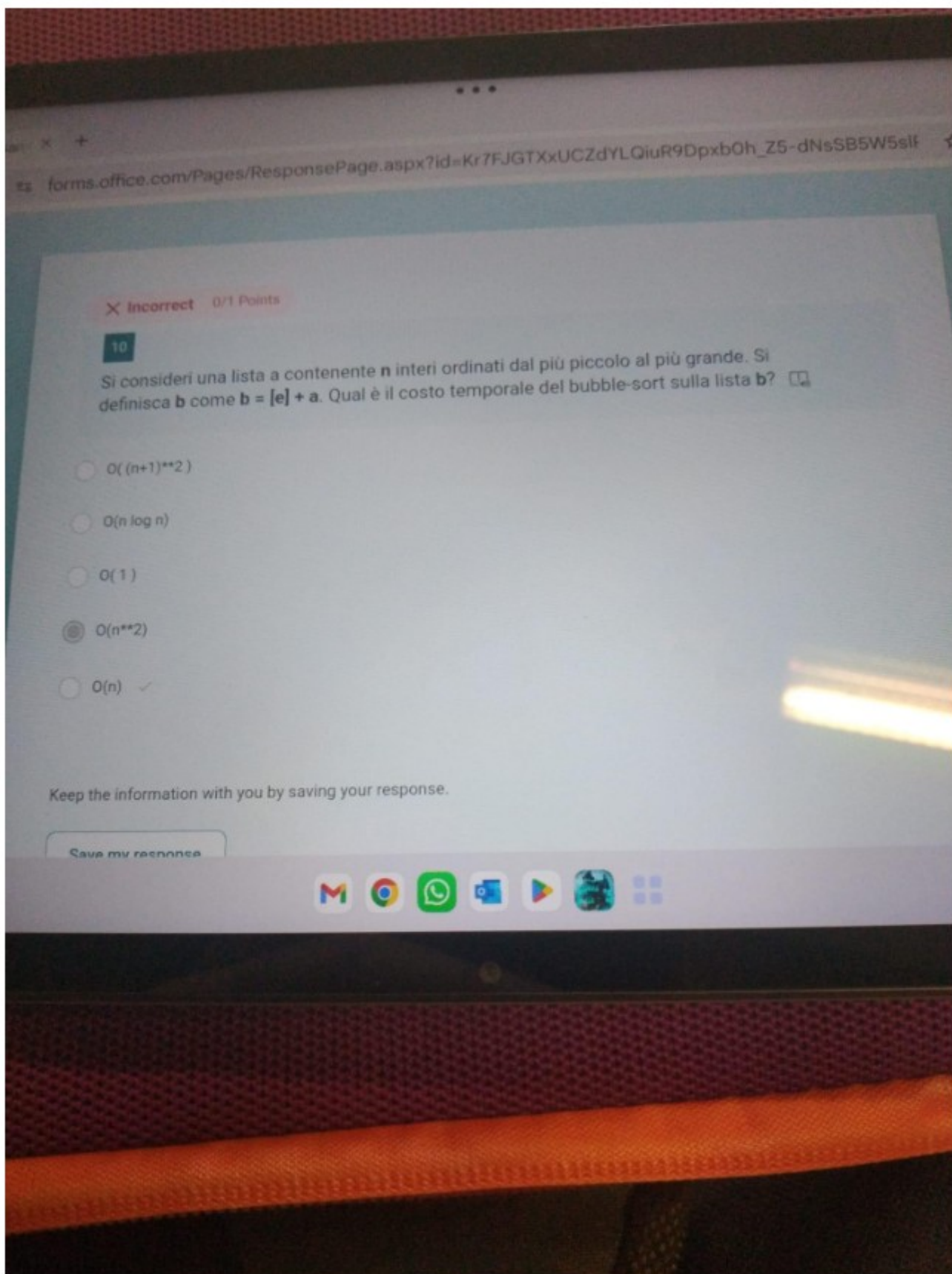
forms.office.com



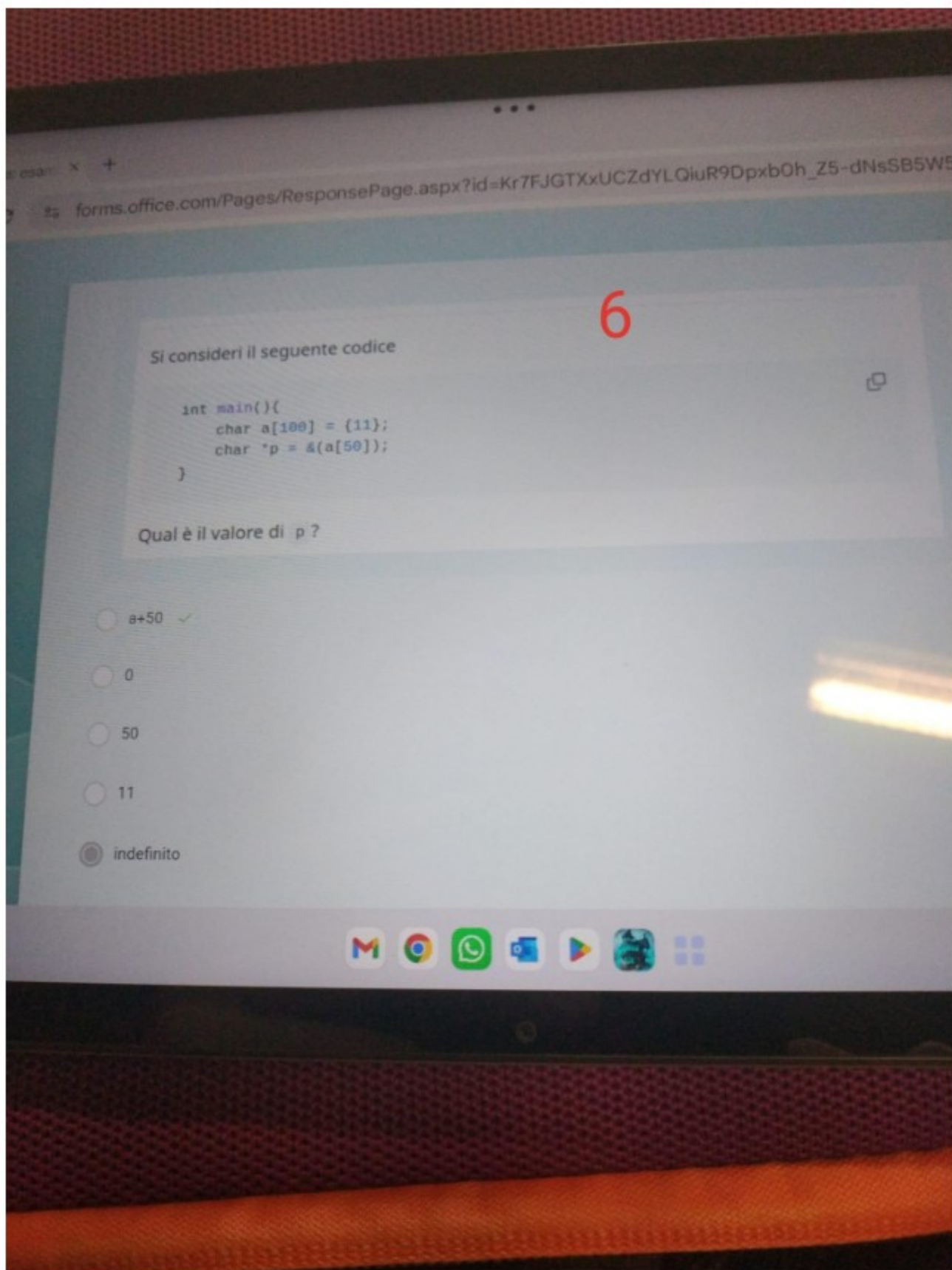
?



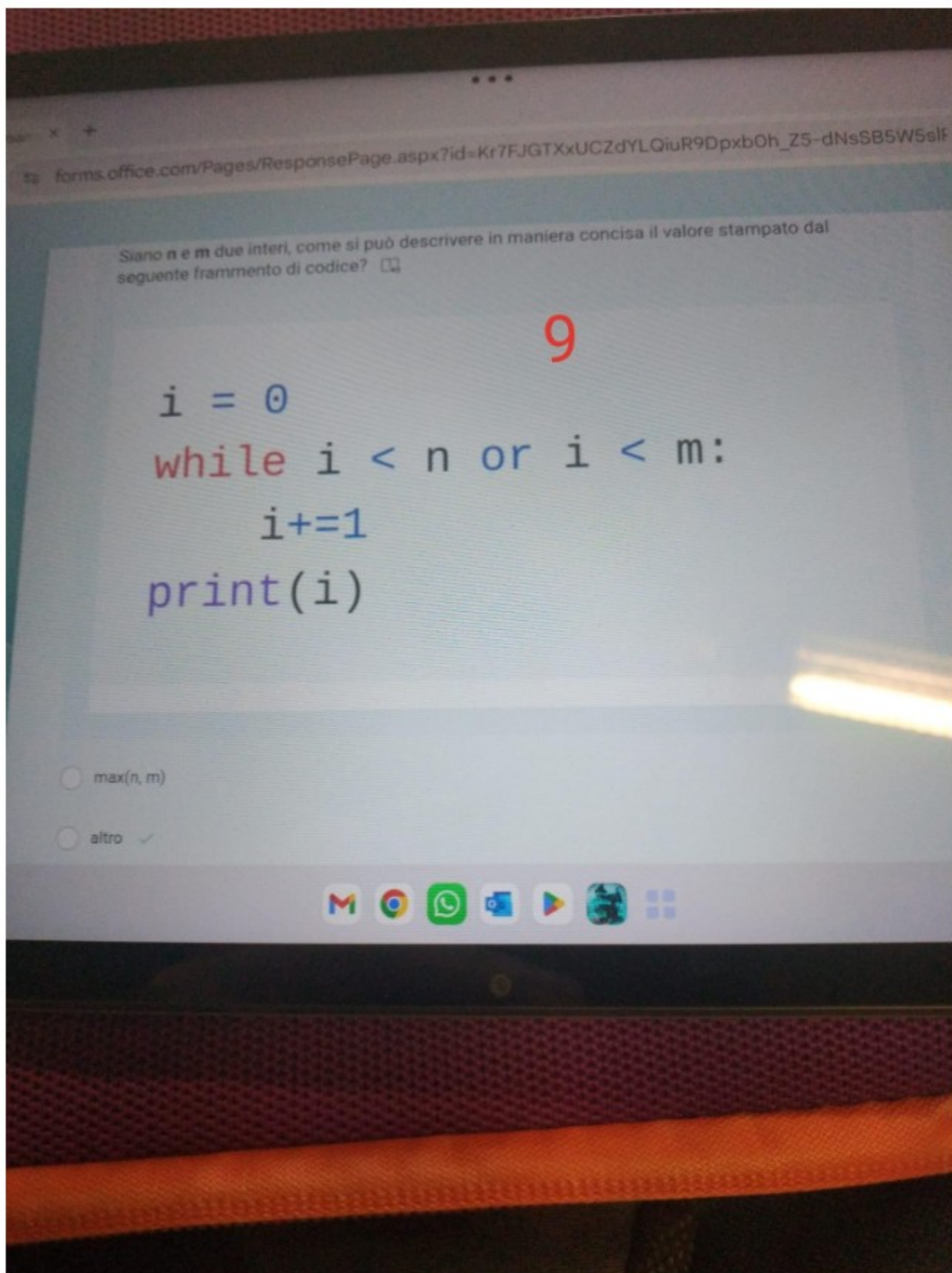
?



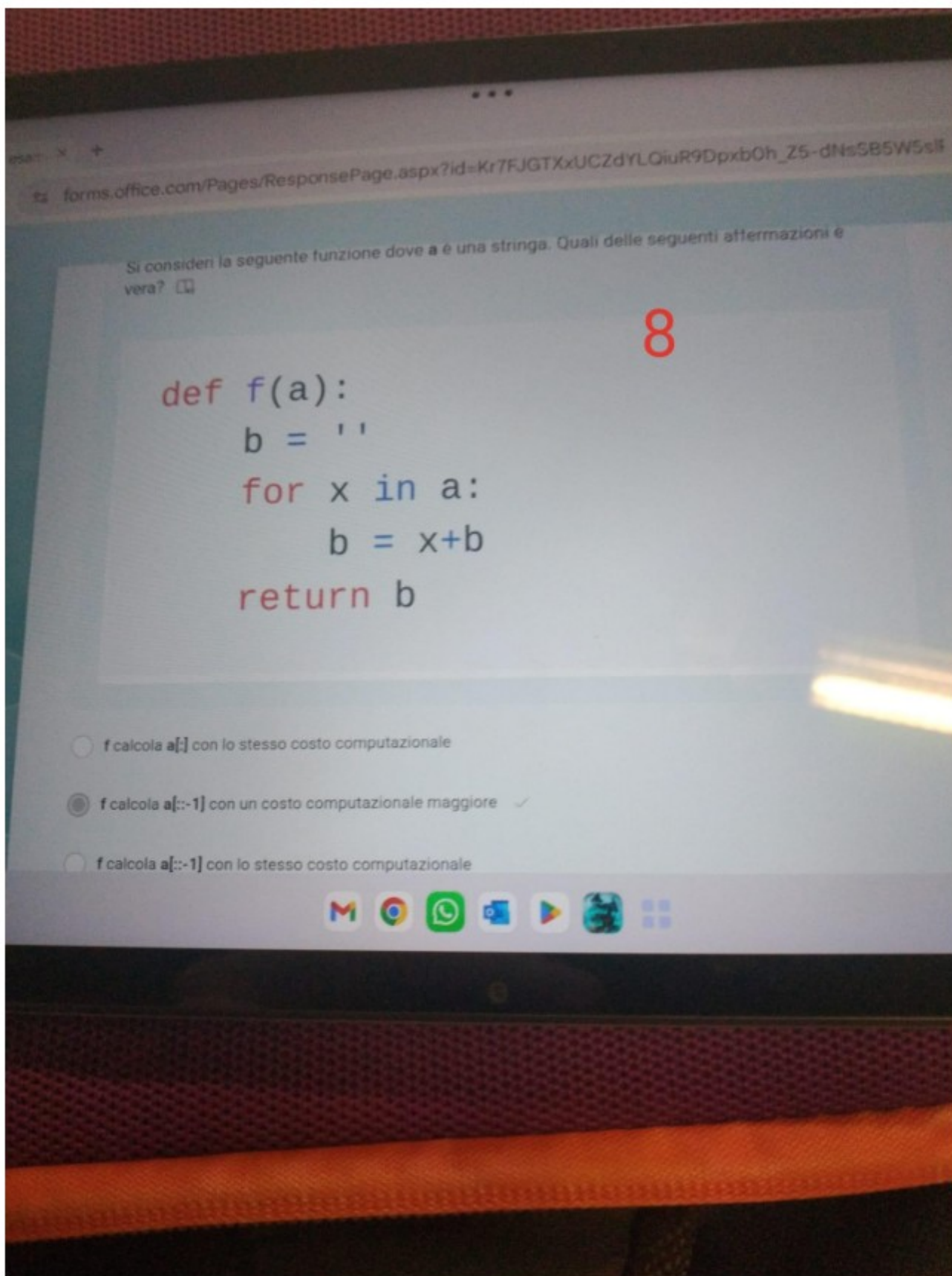
?



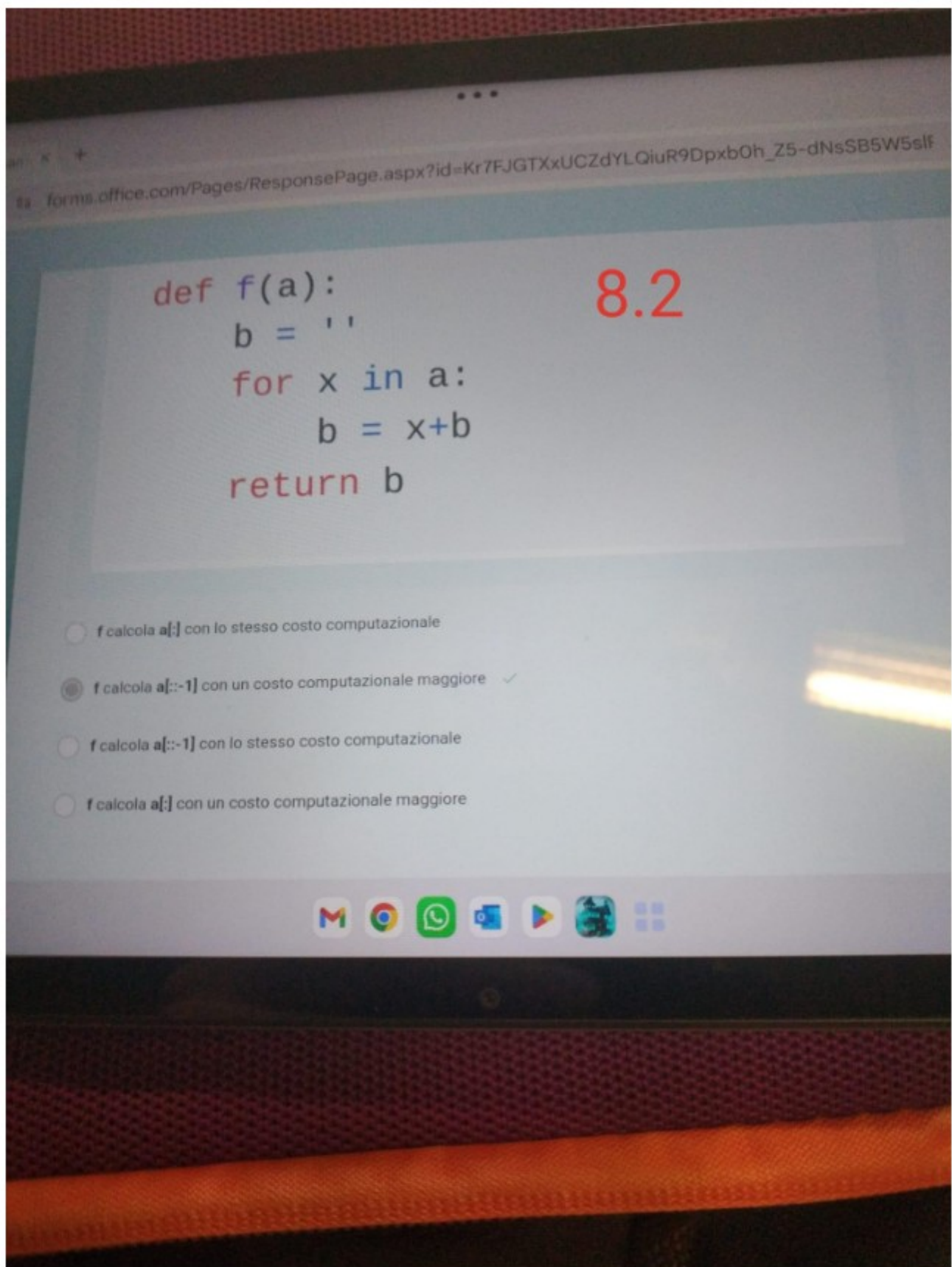
?



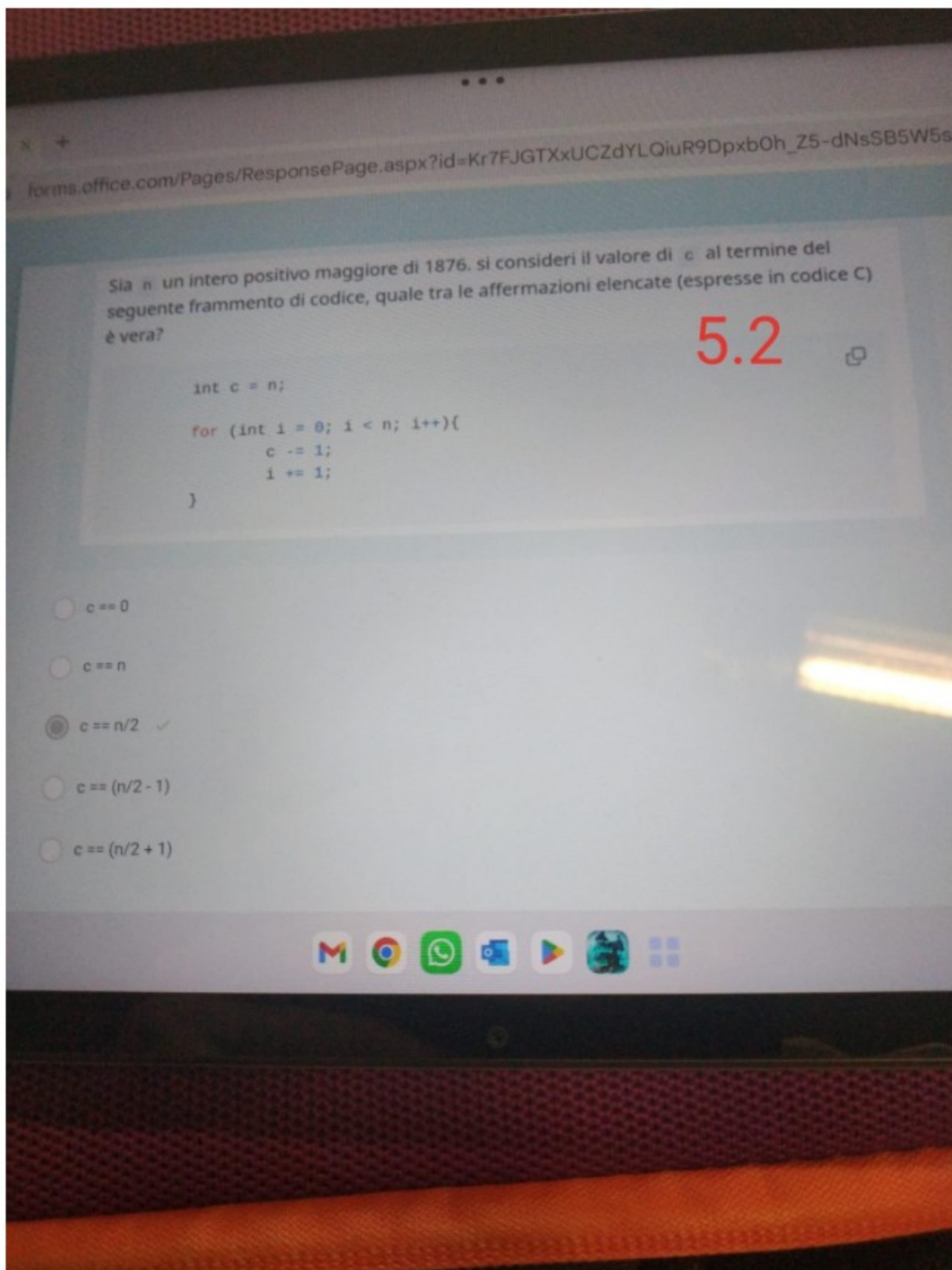
?



?



?



?

forms.office.com/Pages/ResponsePage.aspx?id=Kr7FJGTxxUCZdYLOiuR9Dpxb0h_Z5-dNsSB5W5slf

Sia a una lista di n dizionari ciascuno contenente c elementi a valori `None`. Si consideri il seguente codice:

```
d = {}  
for i in range(len(a)):  
    d = unione(d, a[i])
```

La funzione `unione` ritorna un dizionario che contiene l'unione delle chiavi dei dizionari in input.

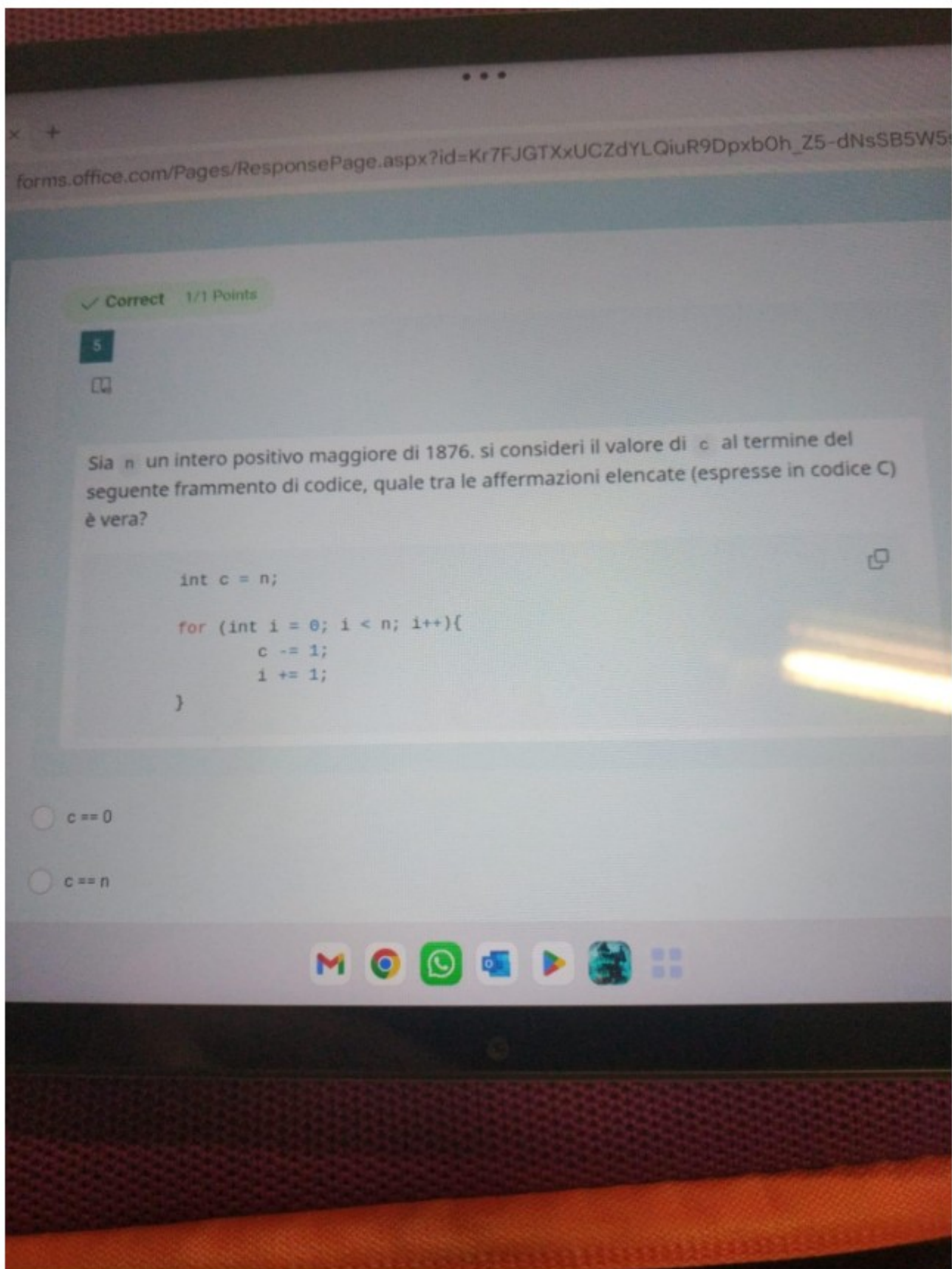
Assumendo che c sia una costante, qual è la complessità temporale del precedente codice?

- ☐ $O(n)$
- ☐ $O(n \log n)$
- ☐ $O(n^{**3})$
- ☐ $O(n^{**2} \log n)$
- ☒ $O(n^{**2})$ ✓

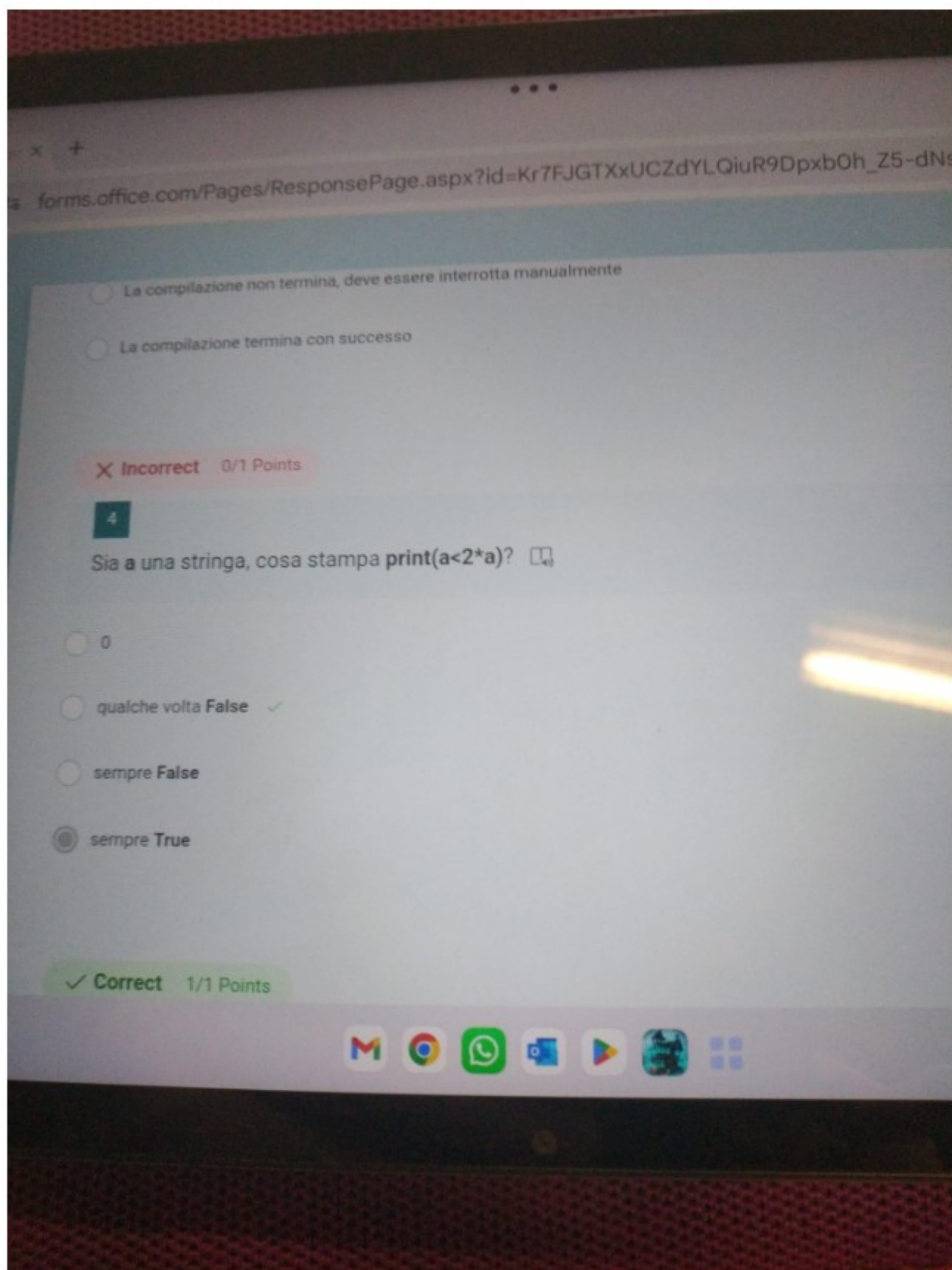
7

Taskbar icons: Gmail, Chrome, WhatsApp, Microsoft Edge, Google Play Store, and a globe icon.

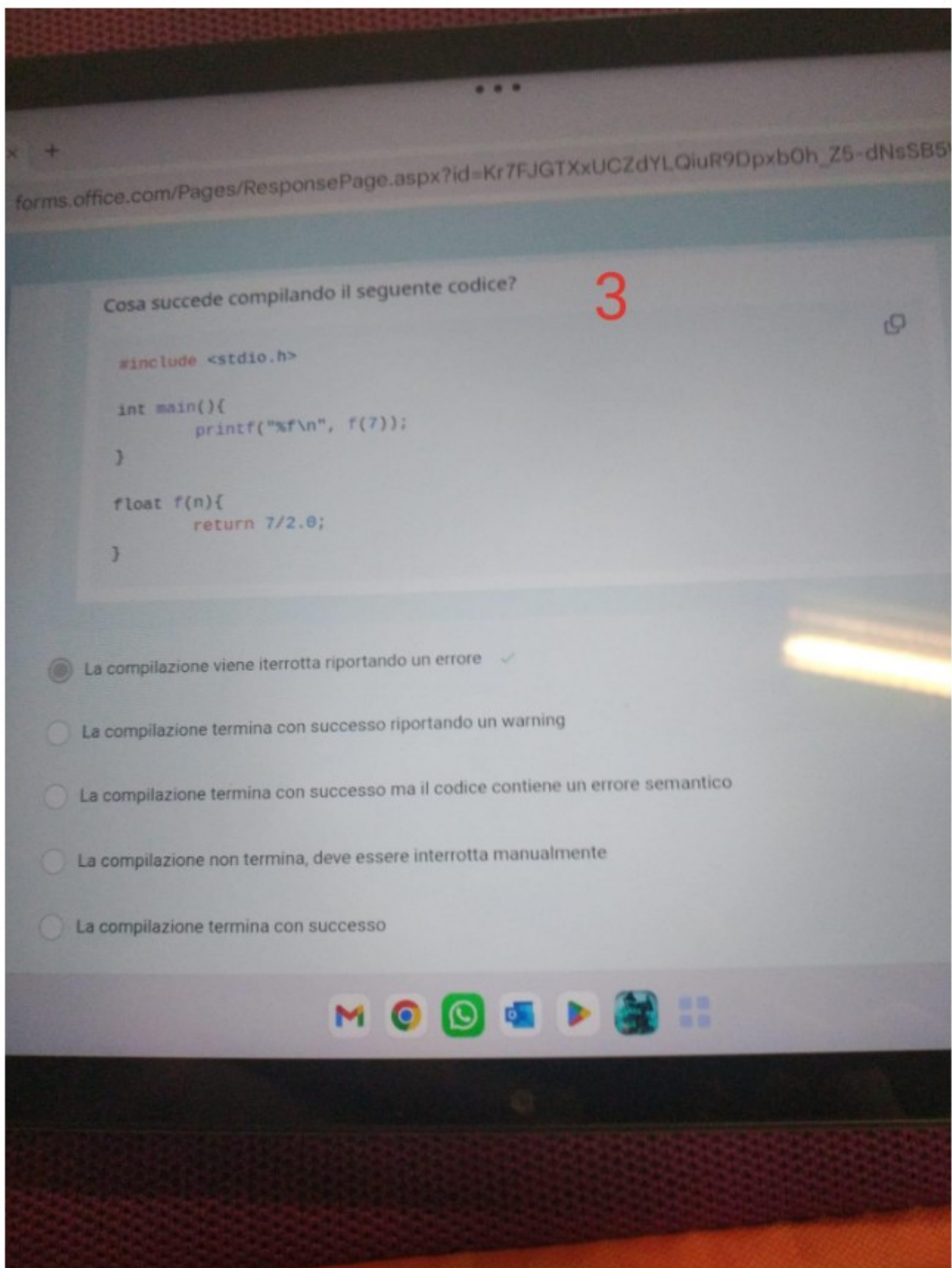
?



?



?

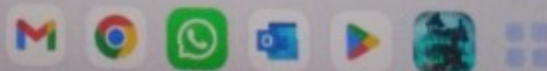


?

2.2

```
str s;  
char x[] = "malloc";  
s.a = x;  
s.n = strlen(x);  
return s;  
}  
  
int main(){  
    str b = new_str();  
    printf("%s\n", b.a);  
    return 6;  
}
```

- ☐ Un ciclo infinito
- ☐ Qualcosa di indefinito ✓
- ☒ Stampa l'intero 6
- ☐ Errore in compilazione
- ☐ Stampa la stringa malloc



?

...
rms.office.com/Pages/ResponsePage.aspx?id=Kr7FJGTxxUCZdYlQiuR9Dpxb0h_Z5-dNsSB5

Cosa succede eseguendo il seguente programma?

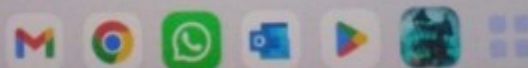
2

```
#include <stdio.h>
#include <string.h>

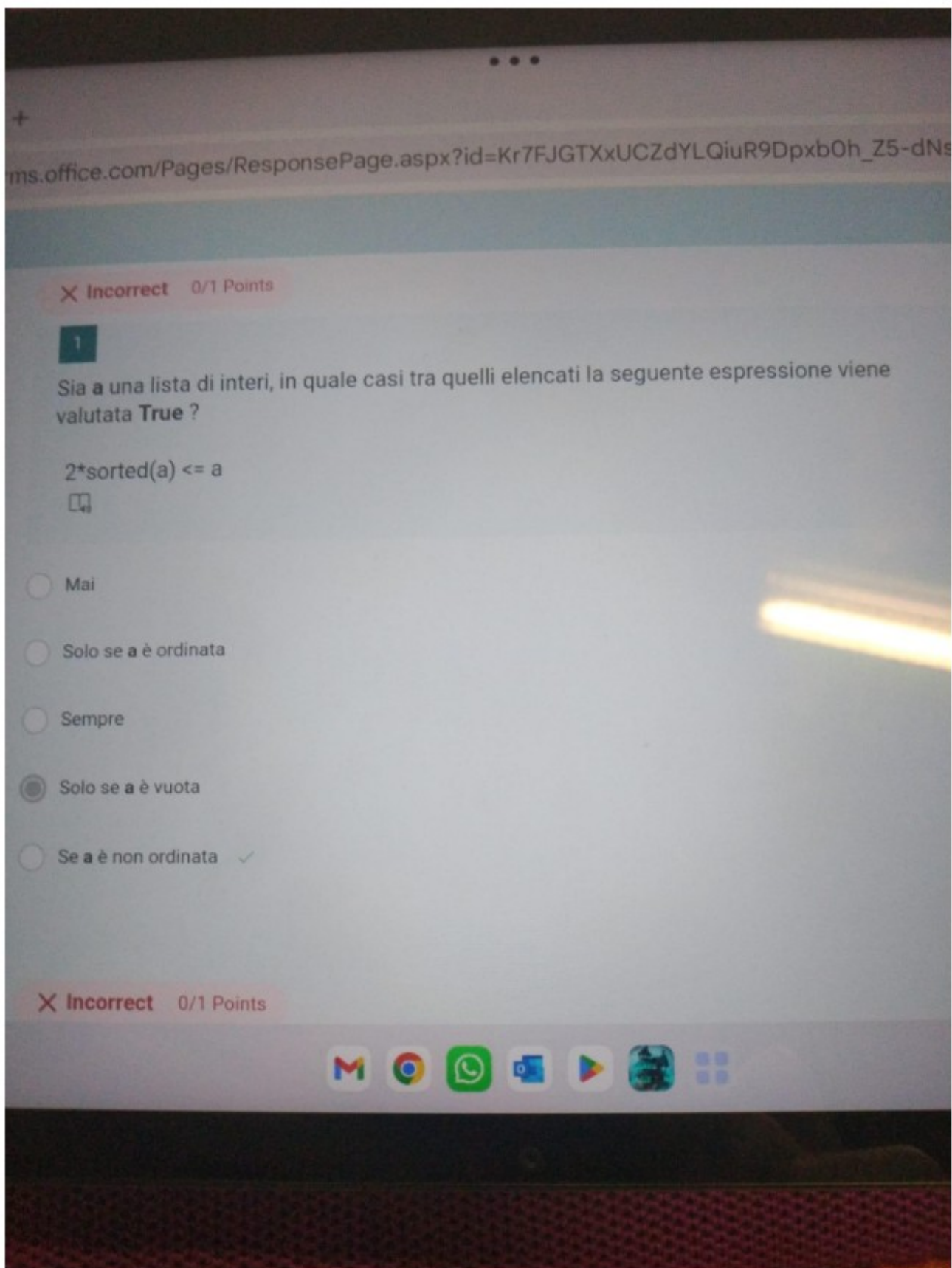
struct str {
    char *a;
    int n;
};
typedef struct str str;

str new_str(){
    str s;
    char x[] = "malloc";
    s.a = x;
    s.n = strlen(x);
    return s;
}

int main(){
    str b = new_str();
    printf("%s\n", b.a);
    return 6;
}
```



?



??

3



In quale caso l'esecuzione di `myfunc(b)` genera un errore?

```
def myfunc( a ):
    n = len(a)
    if n < 2:
        return a
    return myfunc(a[:n//2]+ a[n//2:])
```

- ☐ se `b` è una qualsiasi stringa
- ☐ se `b` è una tupla vuota
- ☐ se il primo elemento di `b` è uno zero
- ☐ mai
- ☒ se `b` è una lista con almeno due elementi ✓

✓ Esatto 1/1 Punti

??

Programmazione. Esame

forms.office.com/Pages/ResponsePage.as

✓ Esatto 1/1 Punti

1

Sia `a` una lista Python di lunghezza `n > 123`, dire quale delle seguenti affermazioni è falsa dopo aver eseguito il seguente frammento di codice.

```
b = a
b.append(n)
a = b + [ 2*n ]
```

☐ `len(b) < max(a)`

☐ `max(a) >= max(b)`

☒ `a[-1] == b[-1]` ✓

☐ `len(a) != len(b)`

☐ `min(a) == min(b)`

✗ Non corretto 0/1 Punti

2

Qual è la complessità spaziale (quantità di memoria supplementare) della funzione `merge` utilizzata dall'algoritmo `merge sort`?

??

✗ Non corretto 0/1 Punti

2



Qual è la complessità spaziale (quantità di memoria supplementare) della funzione `merge` utilizzata dall'algoritmo `merge-sort` ?

- ☐ lineare ✓
- ☐ costante
- ☐ quadratica
- ☒ poli-logaritmica

✓ Esatto 1/1 Punti

3



??

✓ Esatto 1/1 Punti

4



`a` è una lista Python contenente $\sqrt{n}$ interi ordinati in modo crescente seguiti da altri $\sqrt{n}$ interi non ordinati. Qual è il costo del miglior algoritmo che ordini la lista `a` ?

- ☐ $O(n)$
- ☐ $O(n^2)$
- ☐ $O(\sqrt{n})$
- ☒ $O(\sqrt{n} \log n)$ ✓
- ☐ $O(n \log n)$

✗ Non corretto 0/1 Punti

5



??

✖ Non corretto 0/1 Punti

5



Le funzioni `intersect0` e `intersect1` calcolano lo stesso valore, dire quale delle seguenti affermazioni è falsa.

```
def intersect0( a , b):
    for x in a:
        if x in b:
            return True
    return False

def intersect1( a, b ):
    A, B, c = {}, {}, 0
    for x in a:
        A[x] = None
    for x in b:
        B[x] = None

    for x in A.keys():
        if x in B:
            return True

    return False
```

- ☐ Nel caso medio, il tempo di calcolo di `intersect1` è lineare
- ☐ Nel caso medio, il tempo di calcolo di `intersect0` è lineare ✓
- ☐ La memoria supplementare di `intersect1` è lineare
- ☒ La memoria supplementare di `intersect0` è costante
- ☐ Nel caso peggiore, il tempo di calcolo di `intersect0` è quadratico

??



Si consideri il seguente frammento di codice C, quale tra le seguenti affermazioni è vera?

```
int *f( int *k ){
    *k = 3*k;
    return k;
}

void main(){
    int m, n;

    /*
     * ...
     * Operazioni in cui viene assegnato un valore a n
     * ...
     */

    m = *f(&n);
}
```

- ☐ m è 3 volte n
- ☐ m è un array
- ☐ n è 3 volte m
- ☐ n ed m puntano allo stesso indirizzo
- ☒ n e m sono uguali ✓
- ☐ *n è 3 volte *m

✗ Non corretto 0/1 Punti

??

8



Sia N una variabile `int` inizializzata con un valore positivo, qual è il costo computazionale, in termini di tempo di calcolo, del seguente frammento di codice C?

```
int size_a = N;
float *a = malloc(sizeof(float)*size_a);

for (int i = 0; i < 2*N; i++){
    if (i == size_a){
        float *t = a;
        a = malloc(sizeof(float)*(i+1));
        size_a = i+1;
        for(int j = 0; j < size_a; j++)
            a[j] = t[j];
        free(t);
    }
    a[i] = i;
}
```

- ☐ Il tempo di calcolo non dipende da N
- ☐ Sempre lineare in N
- ☐ Lineare in N nel caso medio, quadratico in N nel caso peggiore
- ☐ Lineare in N nel caso peggiore, costante nel caso migliore.
- ☒ Sempre quadratico in N ✓

✗ Non corretto 0/1 Punti

9



✗ Non corretto 0/1 Punti

9



Qual è la complessità temporale (tempo di calcolo) dell'algoritmo merge nel caso in cui le sequenze da fondere abbiano dimensione n e m ?

- ☒ $O(n \log m)$
- ☐ $O(n*m)$
- ☐ $O(\min(n,m))$
- ☐ $O(\max(n,m))$ ✓
- ☐ $O(m \log n)$

✓ Esatto 1/1 Punti

10



Sia L una lista concatenata di n interi implementata nel linguaggio C . Si consideri l'operazione **reset** che prende in input l'indirizzo del primo elemento di L ed un intero k e azzerà il valore dell'elemento in posizione k della lista. Qual è il costo

??

✗ Non corretto 0/1 Punti

7



Sia `a` una lista contenente `bool`, `int` e `str`, cosa restituisce `a_func(a)`?

```
def a_func(a):  
    d = {}  
    for x in a:  
        if x in d:  
            d[x] += 1  
        else:  
            d[x] = 1  
    return max(a, key=lambda x: d[x])
```



- ☒ il numero di volte in cui compare l'elemento più frequente in `a`
- ☐ l'elemento che compare più frequentemente in `a` ✓
- ☐ il tipo che compare più frequentemente in `a`
- ☐ la stringa più lunga in `a`
- ☐ il massimo in `a`

✓ Esatto 1/1 Punti

8



Sia `n` una variabile `int` inizializzata con un valore positivo, qual è il costo computazionale, in termini di tempo di calcolo, del seguente frammento di codice C?

✓ Esatto 1/1 Punti

10



Sia L una lista concatenata di n interi implementata nel linguaggio C . Si consideri l'operazione **reset** che prende in input l'indirizzo del primo elemento di L ed un intero k e azzerà il valore dell'elemento in posizione k della lista. Qual è il costo computazionale in termini di tempo di calcolo della migliore implementazione di **reset**?

- ☐ lineare in n
- ☐ logaritmico in k
- ☐ costante
- ☐ quadratico in n
- ☒ lineare in k ✓

Mantieni le tue informazioni salvando la risposta.

Salva la mia risposta

Torna alla pagina di ringraziamento

8



Si consideri il seguente codice

```
int main(){
    char a[100] = {11};
    char *p = &a[50];
}
```



Qual è il valore di `p`?

☐ 11

https://forms.office.com/Pages/ResponsePage.aspx?id=K7YJSTXGUCZdYUQwRSDp0P_Z5-dHxSB5W5eRvUNFM3TTg1MExRTVg2QkVWU... 5/7

6/19/24, 10:38 AM

Esame 2024-06-19

☒ indefinito

☐ a+50

☐ 50

☐ 0

✓ **Correct** 1/1 Points

9



Sia `n` un intero positivo maggiore di 1876. si consideri il valore di `c` al termine del seguente frammento di codice, quale tra le affermazioni elencate (espresse in codice C) è vera?

```
int c = n;

for (int i = 0; i < n; i++){
    c -= 1;
    i += 1;
}
```



☒ `c == n/2`

☐ `c == (n/2 + 1)`

☐ `c == (n/2 - 1)`

☐ `c == n`

☐ `c == 0`

✗ **Incorrect** 0/1 Points

10

La lista `a` contiene `n` tuple costituite da due elementi ciascuna. Il primo è una etichetta che può valere 0 oppure 1. Si vuole ordinare `a` in modo che le tuple con etichetta 0 precedano quelle con etichetta 1. Qual è il costo temporale del miglior algoritmo che risolve il problema?

☐ $O(1)$

☐ $O(n)$

https://forms.office.com/Pages/ResponsePage.aspx?id=K7YJSTXGUCZdYUQwRSDp0P_Z5-dHxSB5W5eRvUNFM3TTg1MExRTVg2QkVWU... 6/7

6/7

6/19/24, 10:38 AM

Esame 2024-06-19

☐ $O(n^2)$

☒ $O(\log n)$

☐ $O(n \log n)$

Keep the information with you by saving your response.

Save my response

✓ Correct 0/1 Points

5

Cosa succede compilando il seguente codice?

```
#include <stdio.h>

int main(){
    printf("%f\n", f(7));
}

float f(n){
    return 7/2.0;
}
```

- ☐ La compilazione non termina, deve essere interrotta manualmente
- ☐ La compilazione termina con successo
- ☐ La compilazione termina con successo riportando un warning
- ☐ La compilazione termina con successo ma il codice contiene un errore semantico
- ☒ La compilazione viene fermata riportando un errore

✗ Incorrect 0/1 Points

6

Si assuma che **a** e **b** siano due dizionari, qual è la complessità temporale nel caso medio dell'operazione che restituisce la lista delle chiavi in comune tra **a** e **b**?

- ☐ $O(\min(\text{len}(a), \text{len}(b)))$
- ☐ $O(\max(\text{len}(a), \text{len}(b)))$
- ☐ $O(\text{len}(b)/\text{len}(a))$
- ☒ $O(\text{len}(a)*\text{len}(b))$

https://forms.office.com/Pages/ResponsePage.aspx?id=K7TFJSTXUUCZdYUQkRSDpicP_ZC-dHsSD5W5eRr5iUNFM3TTg1MDxRTVq2OkrWU... 4/7

6/19/24, 10:38 AM

Esame 2024-06-19

- ☐ $O(\text{len}(a)/\text{len}(b))$

✗ Incorrect 0/1 Points

7

Sia **n** un intero positivo maggiore di 1876, qual è il valore di **c** al termine del seguente frammento di codice?

```
c = 0
for i in range(n):
    c += 1
    i += 1
```

- ☐ 0
- ☐ n
- ☐ $n/2 - 1$
- ☒ $n/2$
- ☐ $n/2 + 1$

✗ Incorrect 0/1 Points

8

Si consideri il seguente codice

6/19/24, 10:38 AM

Esame 2024-06-19

Cosa succede eseguendo il seguente programma?

```
#include <stdio.h>
#include <string.h>

struct str {
    char *a;
    int n;
};
typedef struct str str;

str new_str(){
    str s;
    char x[] = "malloc";
    s.a = x;
    s.n = strlen(x);
    return s;
}

int main(){
    str b = new_str();
    printf("%s\n", b.a);
    return 0;
}
```

- ☐ Stampa l'intero 6
- ☒ Stampa la stringa malloc
- ☐ Errore in compilazione
- ☐ Un ciclo infinito
- ☐ Qualcosa di indefinito

✗ Incorrect 0/1 Points

4

Si consideri una lista a contenente n interi ordinati dal più piccolo al più grande. Si definisca b come $b = [e] + a$. Qual è il costo temporale del bubble-sort sulla lista b ?

- ☐ $O(1)$
- ☐ $O(n)$
- ☒ $O(n \log n)$
- ☐ $O(n^2)$

6/19/24, 10:38 AM

Esame 2024-06-19

- ☐ $O((n+1)^2)$

✓ Correct 1/1 Points

5

Cosa succede compilando il seguente codice?

```
#include <stdio.h>

int main(){
    printf("%f\n", f(7));
}

float f(n){
    return 7/2.0;
}
```

- ☐ La compilazione non termina, deve essere interrotta manualmente
- ☐ La compilazione termina con successo

Points: 2/10

Incorrect 0/1 Points

1



Sia `a` una lista di `n` dizionari ciascuno contenente `c` elementi a valori `None`. Si consideri il seguente codice:

```
d = {}  
  
for i in range(len(a)):  
    d = unione(d, a[i])
```



La funzione `unione` ritorna un dizionario che contiene l'unione delle chiavi dei dizionari in input.

Assumendo che `c` sia una costante, qual è la complessità temporale del precedente codice?

- ☐ $O(n^3)$
- ☐ $O(2^n)$
- ☐ $O(n^2)$
- ☒ $O(n)$
- ☐ $O(n^2 \log n)$

https://forms.office.com/Pages/ResponsePage.aspx?id=9K7FJ07X0UCZ8YUQkR8Cpe0P_ZS-dHs3B5W5eRvUNFM3TTg1MEvRTVq2QKwU... 1/7

6/19/24, 10:38 AM

Esame 2024-06-19

- ☐ $O(n \log n)$

Incorrect 0/1 Points

2

Sia `a` una lista di interi, in quale caso tra quelli elencati la seguente espressione viene valutata **True**?

`2*sorted(a) <= a`



- ☐ Se `a` è non ordinata
- ☐ Mai
- ☐ Sempre
- ☒ Solo se `a` è vuota
- ☐ Solo se `a` è ordinata

Incorrect 0/1 Points

3



10:17

5G 72

9

Le funzioni `intersect0` e `intersect1` calcolano lo stesso valore, dire quale delle seguenti affermazioni è falsa.

```
def intersect0( a , b):
    for x in a:
        if x in b:
            return True
    return False

def intersect1( a, b ):
    A, B, c = {}, {}, 0
    for x in a:
        A[x] = None
    for x in b:
        B[x] = None

    for x in A.keys():
        if x in B:
            return True

    return False
```

- ☐ La memoria supplementare di `insert0` è costante
- ☐ Nel caso medio, il tempo di calcolo di `insert1` è lineare
- ☐ Nel caso peggiore, il tempo di calcolo di `insert0` è quadratico
- ☒ Nel caso medio, il tempo di calcolo di `insert0` è lineare
- ☐ La memoria supplementare di `insert1` è lineare

22:00 | 0,2KB/s

16%

Programmazione. Esame del 2025-01-20

forms.office.com

✓ Esatto 1/1 Punti

10

Sia `a` una lista Python di lunghezza `n > 123`, dire quale delle seguenti affermazioni è falsa dopo aver eseguito il seguente frammento di codice.

```
b = a
b.append(n)
a = b + [ 2*n ]
```

- ☐ `max(a) >= max(b)`
- ☒ `a[-1] == b[-1]`
- ☐ `min(a) == min(b)`
- ☐ `len(a) != len(b)`
- ☐ `len(b) < max(a)`

10:17

5G 72

9

Le funzioni `intersect0` e `intersect1` calcolano lo stesso valore, dire quale delle seguenti affermazioni è falsa.

```
def intersect0( a , b):
    for x in a:
        if x in b:
            return True
    return False

def intersect1( a, b ):
    A, B, C = {}, {}, {}
    for x in a:
        A[x] = None
    for x in b:
        B[x] = None

    for x in A.keys():
        if x in B:
            return True

    return False
```

- ☐ La memoria supplementare di `insert0` è costante
- ☐ Nel caso medio, il tempo di calcolo di `insert1` è lineare
- ☐ Nel caso peggiore, il tempo di calcolo di `insert0` è quadratico
- ☒ Nel caso medio, il tempo di calcolo di `insert0` è lineare
- ☐ La memoria supplementare di `insert1` è lineare

22:00 | 0,2KB/s

16%

Programmazione. Esame del 2025-01-20

forms.office.com

✓ Esatto 1/1 Punti

10

Sia `a` una lista Python di lunghezza `n > 123`, dire quale delle seguenti affermazioni è falsa dopo aver eseguito il seguente frammento di codice.

```
b = a
b.append(n)
a = b + [ 2*n ]
```

- ☐ `max(a) >= max(b)`
- ☒ `a[-1] == b[-1]`
- ☐ `min(a) == min(b)`
- ☐ `len(a) != len(b)`
- ☐ `len(b) < max(a)`

10:17

5G 72

9

Le funzioni `intersect0` e `intersect1` calcolano lo stesso valore, dire quale delle seguenti affermazioni è falsa.

```
def intersect0( a , b):  
    for x in a:  
        if x in b:  
            return True  
    return False  
  
def intersect1( a, b ):  
    A, B, c = {}, {}, 0  
    for x in a:  
        A[x] = None  
    for x in b:  
        B[x] = None  
  
    for x in A.keys():  
        if x in B:  
            return True  
  
    return False
```

- ☐ La memoria supplementare di `insert0` è costante
- ☐ Nel caso medio, il tempo di calcolo di `insert1` è lineare
- ☐ Nel caso peggiore, il tempo di calcolo di `insert0` è quadratico
- ☒ Nel caso medio, il tempo di calcolo di `insert0` è lineare ✓
- ☐ La memoria supplementare di `insert1` è lineare

22:00 | 0,2KB/s

16%

Programmazione. Esame del 2025-01-20

forms.office.com

✓ Esatto 1/1 Punti

10

Sia `a` una lista Python di lunghezza `n > 123`, dire quale delle seguenti affermazioni è falsa dopo aver eseguito il seguente frammento di codice.

```
b = a  
b.append(n)  
a = b + [ 2*n ]
```

- ☐ `max(a) >= max(b)`
- ☒ `a[-1] == b[-1]` ✓
- ☐ `min(a) == min(b)`
- ☐ `len(a) != len(b)`
- ☐ `len(b) < max(a)`

10:17

5G 72

✓ Esatto 1/1 Punti

7

Sia L una lista concatenata di n interi implementata nel linguaggio C. Si consideri l'operazione `reset` che prende in input l'indirizzo del primo elemento di L ed un intero k e azzerà il valore dell'elemento in posizione k della lista. Qual è il costo computazionale in termini di tempo di calcolo della migliore implementazione di `reset`?

☒ lineare in k ✓

☐ costante

☐ logaritmico in k

☐ quadratico in n

☐ lineare in n

✓ Esatto 1/1 Punti

8

a è una lista Python contenente $\sqrt{n}$ interi ordinati in modo crescente seguiti da altri $\sqrt{n}$ interi non ordinati. Qual è il costo del miglior algoritmo che ordini la lista a ?

☐ lineare in n

✓ Esatto 1/1 Punti

8

a è una lista Python contenente $\sqrt{n}$ interi ordinati in modo crescente seguiti da altri $\sqrt{n}$ interi non ordinati. Qual è il costo del miglior algoritmo che ordini la lista a ?

☐ $O(n)$

☐ $O(n^2)$

☐ $O(n \log n)$

☐ $O(\sqrt{n})$

☒ $O(\sqrt{n} \log n)$ ✓

10:17

5G 72

5

Si consideri il seguente frammento di codice C, quale tra le seguenti affermazioni è vera?

```
int *f( int *k ){
    *k = 3*k;
    return k;
}

void main(){
    int n,m;

    /*
     * ...
     * operazioni in cui viene assegnato un valore a n
     * ...
     */

    m = *f(&n);
}
```

- ☐ n e m sono uguali ✓
- ☐ m è 3 volte n
- ☐ n è 3 volte m
- ☐ *n è 3 volte *m
- ☒ n ed m puntano allo stesso indirizzo
- ☐ m è un array

22:00 | 0,5KB/s

16%

Programmazione. Esame del 2025-01-20
form.office.com

✓ Esatto 1/1 Punti

6

In quale caso l'esecuzione di myfunc(b) genera un errore?

```
def myfunc( a ):
    n = len(a)
    if n < 2:
        return a
    return myfunc(a[:n//2]+a[n//2:])
```

- ☐ mai
- ☐ se b è una qualsiasi stringa
- ☐ se il primo elemento di b è uno zero
- ☒ se b è una lista con almeno due elementi ✓
- ☐ se b è una tupla vuota

✓ Esatto 1/1 Punti

10:17

5G 72

Sia N una variabile `int` inizializzata con un valore positivo, qual è il costo computazionale, in termini di tempo di calcolo, del seguente frammento di codice C?

```
int size_a = N;
float *a = malloc(sizeof(float)*size_a);

for (int i = 0; i < 2*N; i++){
    if (i == size_a){
        float *t = a;
        a = malloc(sizeof(float)*(i+1));
        size_a = i+1;
        for(int j = 0; j < size_a; j++){
            a[j] = t[j];
        }
        free(t);
    }
    a[i] = i;
}
```

- ☐ Il tempo di calcolo non dipende da N
- ☐ Lineare in N nel caso peggiore, costante nel caso migliore.
- ☒ Lineare in N nel caso medio, quadratico in N nel caso peggiore
- ☐ Sempre quadratico in N . ✓
- ☐ Sempre lineare in N

✓ Esatto 1/1 Punti

22:00 | 0,0KB/s

16%

Programmazione. Esame del 2025-01-20

forno.office.com

- ☐ Sempre quadratico in N . ✓
- ☐ Sempre lineare in N

✓ Esatto 1/1 Punti

4

Qual è la complessità temporale (tempo di calcolo) dell'algoritmo `merge` nel caso in cui le sequenze da fondere abbiano dimensione n e m ?

- ☐ $O(\min(n,m))$
- ☒ $O(\max(n,m))$ ✓
- ☐ $O(n*m)$
- ☐ $O(m \log n)$
- ☐ $O(n \log m)$

10:17

5G 72

✓ Esatto 1/1 Punti

1

Qual è la complessità spaziale (quantità di memoria supplementare) della funzione `merge` utilizzata dall'algoritmo `merge-sort`?

- ☒ lineare ✓
- ☐ poli-logaritmica
- ☐ costante
- ☐ quadratica

✓ Esatto 1/1 Punti

2

21:59 | 5,2KB/s

Programmazione. Esame del 2025-01-20

forms.office.com

- ☐ quadratica

✓ Esatto 1/1 Punti

2

Sia `a` una lista contenente `bool`, `int` e `str`, cosa restituisce `a_func(a)`?

```
def a_func(a):
    d = {}
    for x in a:
        if x in d:
            d[x] += 1
        else:
            d[x] = 1
    return max(d, key=lambda x: d[x])
```

- ☒ l'elemento che compare più frequentemente in `a` ✓
- ☐ la stringa più lunga in `a`
- ☐ il numero di volte in cui compare l'elemento più frequente in `a`
- ☐ il massimo in `a`
- ☐ il tipo che compare più frequentemente in `a`

✗ Non corretto 0/1 Punti

3

10:15

5G 73

✓ Esatto 1/1 Punti

7

Assumendo che un puntatore occupi 8 byte ed un int 4 byte, quale delle seguenti

22:10 | 0,2KB/s

Esame 2024-09-16

forms.office.com

✓ Esatto 1/1 Punti

5

Cosa ritorna `enigma('01234567')`?

```
def enigma(a):
    if len(a) <= 2:
        return a
    else:
        return enigma(a[len(a)//2:]) +
               enigma(a[:len(a)//2])
```

☐ 76540123

☐ 01234567

☐ 76543210

☒ 67452301

☐ 10325476

✓ Esatto 1/1 Punti

6

Sia `n` un intero positivo maggiore di 1024, qual è la complessità temporale dell'ultima riga del seguente frammento di codice C?

22:10 | 5,2KB/s

Esame 2024-09-16

forms.office.com

✗ Non corretto 0/1 Punti

4

Si consideri la seguente operazione di assegnazione

`x0, x1, ... = y0, y1,`

dove a sinistra dell'uguale

10:15

5G 73

8

Qual è il valore di 'b' alla fine del seguente frammento di codice?

```
a = [4, 3, 2, 1]

for x in a:
    try:
        b += x
    except NameError:
        b = 1
```

☐ NameError

☐ 7

☐ 1

☐ 6

☒ 10

☐ 4

✗ Non corretto 0/1 Punti

9

22:10 | 1.4KB/s

20%

Esame 2024-09-16

forms.office.com

☐ peggiore

✓ Esatto 1/1 Punti

7

Assumendo che un puntatore occupi 8 byte ed un int 4 byte, quale delle seguenti affermazioni è vera dopo l'esecuzione del seguente frammento di codice?

```
int *a = malloc(2*sizeof(a));
int b[8] = {0};

int size_a = sizeof(a);
int size_b = sizeof(b);
```

☐ 'size_b == 2*size_a'

☒ 'size_b == 4*size_a'

☐ 'size_b == size_a'

☐ 'size_b == 4'

10:14

5G 73

✗ Non corretto 0/1 Punti

12

Siano i e n interi positivi con $i < n$,
quanto vale $a[i]$ al termine del seguente
frammento di codice?

```
int i, *a;

for (i = 0; i < n; i++){
    if (i%10 == 0){
        a = malloc((i+1)*10*sizeof(int));
    }
    a[i] = i;
}
```

- ☐ altro
- ☐ 0
- ☐ i solo se $n < 10$
- ☐ indefinito sempre

22:10 | 6,5KB/s

21%

Esame 2024-09-16

forma.office.com

- ☐ $O(\min(n^2, n \log n))$
- ☐ $O(n)$

✓ Esatto 1/1 Punti

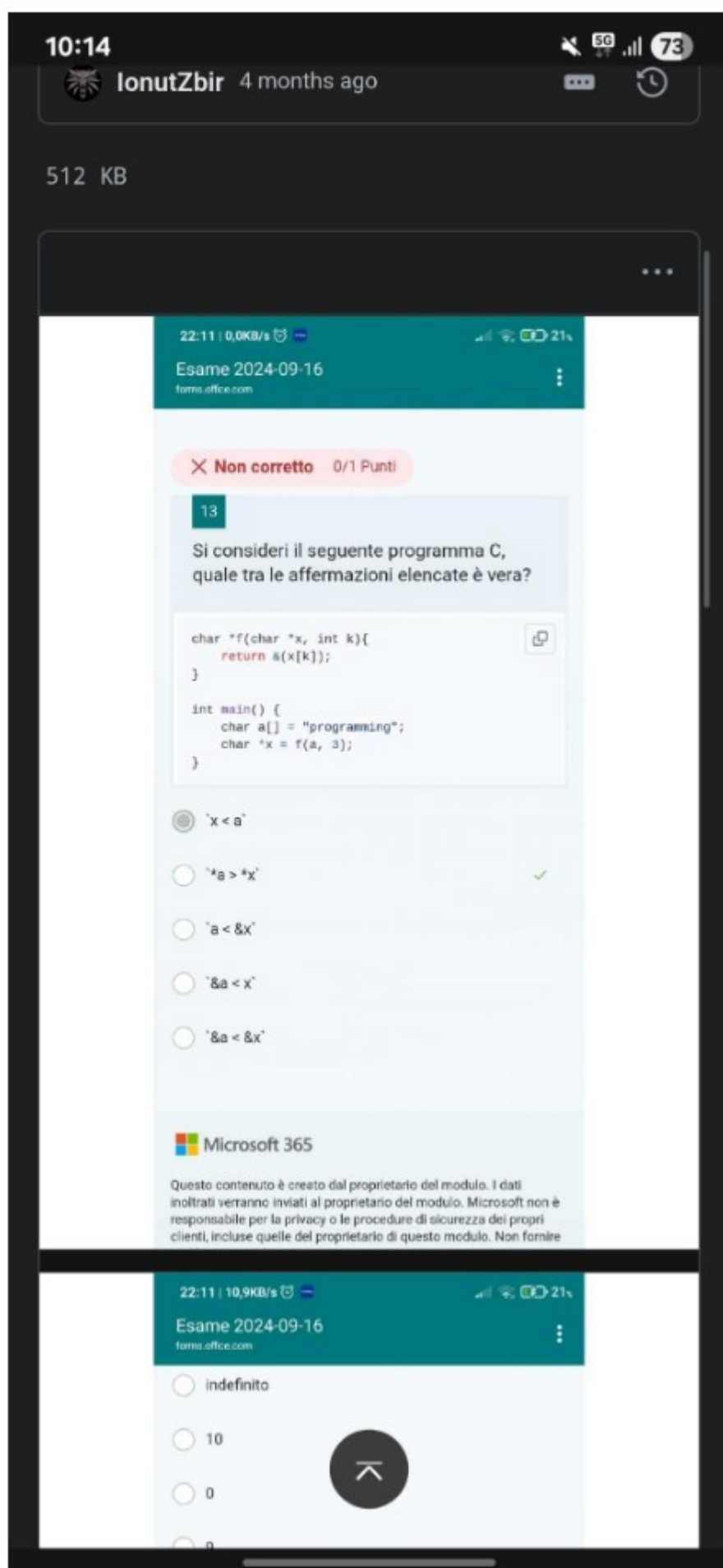
11

Qual è il valore della variabile n al termine
del seguente frammento di codice C?

```
int n, a = 0;
int *b = &a;
a++;
n = *b+1;
```

- ☒ 11
- ☐ indefinito
- ☐ 10
- ☐ 0
- ☐ 9

✗ Non corretto 0/1 Punti



Compilation

10:17

5G

72

✓ Esatto

1/1 Punti

10

x, y: sono due variabili Python di tipo int. In quale caso la condizione `x % y == 0` è vera?

☐ mai

☒ se `y == x+1`

☐ se `x == 1`

☐ sempre

☐ se x ed y sono primi

Questo contenuto è creato dal proprietario del modulo. I dati raccolti verranno inviati al proprietario del modulo. Microsoft non è responsabile per la privacy o le procedure di sicurezza dei propri clienti, incluse quelle del proprietario di questo modulo. Non fornire mai la password.

Con tecnologia Microsoft Forms | [Privacy e cookie](#) | [Condizioni per l'utente](#) | [Accessibilità](#)

Compilation

10:17

5G 72

Siano a e b due liste di lunghezza n . a contiene interi, b contiene stringhe. Siano i e j interi positivi minori di n , diversi tra di loro. Si consideri il seguente codice, quale delle seguenti affermazioni è sempre vera?

```
a[j] = b  
a = a[i]  
a[j] = 'a'
```

- ☐ $b[k] == c[j][k]$
- ☒ $a[j][k]$ è un int
- ☐ $a[k] == b[k]$
- ☐ $c[j][k] != a[j][k]$

✓ Esatto 1/1 Punti

8

Da d un intero positivo maggiore di 1 e n la struttura derivata dal seguente codice Python. Quale delle seguenti affermazioni è sempre vera?

```
d = []  
for i in range(n):  
    d.append(d.append(d) + 1)
```

- ☐ $\text{len}(d) < n$
- ☐ $d[0] == d[1]$
- ☐ $\text{len}(d) == n$
- ☒ $d[0] > d[1]$

✗ Non corretto 0/1 Punti

9

Si consideri il seguente codice C:

```
union {  
    int *u;  
    float *f;  
    double *d;  
} u;  
  
struct {  
    int *u;  
    float *f;  
} s;  
  
int size_u = sizeof(u);  
int size_s = sizeof(s);
```

Quale tra le seguenti affermazioni è sempre vera?

- ☐ $\text{size}_s == 2 * \text{size}_u$
- ☐ $\text{size}_u == \text{size}_s$
- ☐ $\text{size}_u == 1.5 * \text{size}_s$
- ☒ $\text{size}_u > \text{size}_s$

✓ Esatto 1/1 Punti

10

Compilation

10:16

5G 72

Sia a una lista di interi e siano i e j due indici di a ; quale tra le seguenti affermazioni è sempre vera dopo l'esecuzione del seguente codice?

```
b = sorted(a, key=lambda x: (1 if x == a[i] else 0, x))
```

- ☐ $b[i] \leq b[j]$
- ☐ $a[i] \leq a[j]$
- ☒ se $b[i] < 0$ e $b[j] > 0$ allora $b[i] > b[j]$
- ☐ se $a[i] < 0$ e $a[j] > 0$ allora $b[i] > b[j]$

✓ Esatto 1/1 Punti

5

Sia n un intero positivo ed q la stringa definita dal seguente frammento di codice; quanto vale $\text{len}(q)$?

```
def f(x) = null() if (x < 0) else f(x-1);
```

- ☐ $n-1$
- ☒ infinito
- ☐ 0
- ☐ n

✓ Esatto 1/1 Punti

6

Sia L una lista concatenata implementata in C; ogni nodo contiene un intero ed il puntatore al nodo successivo oppure NULL ; se h è l'ultimo nodo, q è una variabile che contiene il puntatore al primo nodo della lista; il numero di elementi della lista ed il puntatore al nodo contenente il minimo elemento nella lista.

Quale sarebbe la complessità temporale di una funzione efficiente che invoca la lista?

- ☐ Lineare nel caso peggiore, costante nel caso migliore
- ☐ Quadratica nel caso medio
- ☐ Lineare nel caso medio, quadratica nel caso peggiore
- ☒ Lineare sempre

✗ Non corretto 0/1 Punti

7

Siano a e b due liste di lunghezza n ; a contiene interi, b contiene stringhe. Siano i e j interi positivi minori di n tra di loro. Si consideri il seguente codice; quale delle seguenti affermazioni è sempre vera?

```
a[j] = b
```

Compilation

10:16

5G 72

✗ Non corretto 0/1 Punti

1

a e b sono due listoni di dimensioni rispettivamente n_a ed n_b . Qual è il costo nel caso medio di un algoritmo efficiente che computa la lista delle distanze che sono sia di a che di b ?

- ☐ $O(\min(n_a, n_b))$
- ☐ $O(n_a + n_b)$
- ☐ $O(\max(n_a, n_b))$
- ☒ $O(n_a * n_b)$

✓ Esatto 1/1 Punti

2

Si consideri il seguente codice Python, assumendo che a contenga stringhe e interi, e quale caso l'ordine di a è positivo?

```
def a(a):  
    for i in range(len(a)):  
        a[i] = a[i] + 1  
    return a
```

- ☒ Sempre
- ☐ Se il primo elemento di a è intero
- ☐ Mai

- ☐ Se gli interi sono più delle stringhe

✓ Esatto 1/1 Punti

3

Dia a un intero positivo maggiore di 1000, qual è la complessità temporale della linea riga del seguente frammento di codice?

```
def f(a):  
    for i in range(1, a+1):  
        for j in range(1, a+1):  
            a = a + 1  
    return a
```

- ☒ costante nel caso peggiore
- ☐ lineare nel caso peggiore
- ☐ costante nel caso medio, lineare nel caso peggiore
- ☐ lineare nel caso medio
- ☐ quadratico nel caso peggiore

✓ Esatto 1/1 Punti

4

Dia a una lista di interi e siano i e j due indici di a , quale tra le seguenti affermazioni è sempre vera dopo l'esecuzione del seguente codice?

```
b = sorted(a, key=lambda x: 1 if x < 0 else 0, reverse=True)
```

- ☐ $b[i] \leq b[j]$

10:13

5G 73

https://forms.office.com/Pages/ResponsePage.aspx?id=9f7fJG7XUJC26YUQwR8Qp0R_Z5-dN43B9W5sRvUNES4TUNCVFNSRvZzQ7SL... 9/10

09/03/23, 12:02

Prova Intermedia 2 - 2023-03-08

✓ Esatto 1/1 Punti

10

In quali casi la funzione `f` restituisce `None` ?

```
def f(a, b):
    try:
        if a < b:
            return a+b
        else:
            return True
    except TypeError:
        return None
```

- ☐ se `a` e `b` sono stringhe
- ☐ se `a` e `b` sono numeri
- ☒ se `a` è un numero e `b` una stringa
- ☐ se `a` e `b` sono liste
- ☐ mai

[Torna alla pagina di ringraziamento](#)

Questo contenuto è creato dal proprietario del modulo. I dati inoltrati verranno inviati al proprietario del modulo. Microsoft non è responsabile per la privacy o le procedure di sicurezza dei propri clienti, incluse quelle del proprietario di questo modulo. Non fornire mai la password.

Con tecnologia Microsoft Forms | [Privacy e cookie](#) | [Consentimenti all'utilizzo](#) | [Accessibilità](#)

https://forms.office.com/Pages/ResponsePage.aspx?id=9f7fJG7XUJC26YUQwR8Qp0R_Z5-dN43B9W5sRvUNES4TUNCVFNSRvZzQ7SL... 10/10



10:13



Per quali dei seguenti tipi per `a` e `b` la funzione `f` non genera

```
def f(a, b):
    c = a+b
    c.sort()
    return c[0]
```

- ☐ a lista, b tupla
- ☐ a tupla, b tupla
- ☒ a lista, b lista
- ☐ a tupla, b lista

https://forms.office.com/Pages/ResponsePage.aspx?id=R07FJ07XUJCZ6YUQaR9Qp0dH_Z5-8Nu88W5aRvVINE&TUNCVFN8RvZsOT&L... 8/19

08/03/23, 12:02

Prova Intermedia 2 - 2023-03-08

✗ Non corretto 0/1 Punti

9

Sia `a` una lista di interi, cosa contiene la lista `c` così definita?

```
c = [a[i]/(1+i) for i in range(len(a))]
```

- ☐ tutti 0
- ☐ tutti float
- ☒ almeno un int
- ☐ tutti int



10:13

5G 73

Sia `a` una lista di stringhe, cosa indica `c` al termine del seguente frammento di codice?

```
c = 0
for x, y in zip(a[:-1], a[1:]):
    c += 1 if x < y else 0
```

- ☐ il numero di stringhe nella prima parte di `a` più piccole di quelle nella seconda parte
- ☐ il numero di sottostringhe contenute in `a`
- ☐ il numero di elementi in `a` che precedono il loro successivo
- ☒ il numero di stringhe in `a` più corte della successiva

https://forms.office.com/Pages/ResponsePage.aspx?id=4u7fJ37XUJCZ6YlQwR8Dp00H_Z5-dh4d8ROVdsRvUNE5TUNCVFN3RvZ3Q1dL... 6/10

08/03/23, 12:02

Prova Intermedia 2 - 2023-03-08

✓ Esatto 1/1 Punti

7

Sia `n` un intero positivo, quante liste vengono create dal seguente frammento di codice?

```
a = [i**2 for i in range(n)]
b = a
c = b[1:]
```

- ☐ Una
- ☒ Due
- ☐ Nessuna
- ☐ Tre



10:13

5G 74

✗ Non corretto 0/1 Punti

4

Siano `a` e `b` due liste di lunghezza `n`. `a` contiene interi, `b` contiene stringhe. Siano `k` e `j` interi positivi minori di `n` diversi tra di loro. Consideri il seguente codice, quale delle seguenti affermazioni è vera?

```
a[k] = b
c = a[:]
c[k][j] = 1
a[j] = 'one'
```

- ☐ `c[k][j] == a[k][j]`
- ☐ `c[j]` è un `str`
- ☒ `c[k][j] != a[k][j]`
- ☐ `a[j] == c[j]`

https://forms.office.com/Pages/ResponsePage.aspx?id=4Y7FJG7XUJC26Y1Q6R8Qp60P_Z5-8N4SRW5xRvU1ME5TUNCVTNSRvZsQTE... 4/10

08/03/23, 12:02

Prova Intermedia 2 - 2023-03-08

✓ Esatto 1/1 Punti

5

Sia `d` un dizionario, cosa restituisce `d.get(10, 10.5)` ?

- ☐ una chiave del dizionario
- ☒ un valore del dizionario o un `float`
- ☐ un valore del dizionario o un'eccezione
- ☐ un valore del dizionario o `None`
- ☐ un valore del dizionario o un intero



10:13

5G 74

✓ Esatto 1/1 Punti

3

Qual è il costo computazionale in termini di tempo di calcolo del miglior algoritmo che ordini gli elementi di 3 sequenze di lunghezza n già ordinate?

- ☐ cubico in n
- ☒ lineare in n
- ☐ polilogaritmico in n
- ☐ quadratico in n

https://forms.office.com/Pages/ResponsePage.aspx?id=907F307XUJCZ6YUQaR9DpK0H_Z5-dNz00W5oRvUNE3TUNCVFNSRvZzQTSL... 3/10

08/03/23, 12:02

Prova intermedia 2 - 2023-03-08

✗ Non corretto 0/1 Punti

4

Siano a e b due liste di lunghezza n . a contiene interi, b contiene stringhe. Siano k e j interi positivi minori di n diversi tra di loro. Consideri il seguente codice, quale delle seguenti affermazioni è vera?

```
a[k] = b
c = a[:j]
c[k][j] = 1
a[j] = 'one'
```

- ☐ $c[k][j] == a[k][j]$
- ☐ $c[j]$ è un `str`
- ☐ $c[k][j] != a[k][j]$



10:13

5G 74

✓ Esatto 1/1 Punti

1

Sia `a` una lista di interi, cosa restituisce `max(a, key=str)` ?

- ☐ L'intero più grande
- ☒ Se c'è, un intero che inizia con 9
- ☐ Viene generato un `TypeError`
- ☐ L'intero con più cifre

https://forms.office.com/Pages/ResponsePage.aspx?id=6U7FJG7XGJCZ6Y1QaR0Qp0H_Z5-d4d3BVA6RvUNE5tUNCVFN2RzZaQ5L... 1/10

08/03/23, 12:02

Pizza intermedia 2 - 2023-03-08

✓ Esatto 1/1 Punti

2

Per quale tipo di input la funzione `f` genera un'eccezione?

```
def f( a ):
    if len(a) == 0:
        return ''
    else:
        return str(a[0])+f(a[1:])
```

- ☒ dict
- ☐ tuple
- ☐ str
- ☐ list

10:16



lineare in tutti i casi

✓ **Esatto** 1/1 Punti

9

Sia **L** una lista concatenata implementata in C: ogni nodo contiene un intero ed il puntatore al nodo successivo oppure **NULL** se è l'ultimo nodo; **L** è una **struct** che contiene il puntatore al primo nodo della lista; il numero di elementi della lista ed il puntatore al nodo contenente in minimo elemento nella lista.

Quale sarebbe la complessità temporale di una funzione efficiente che elimina il nodo contenente il minimo? [?]

- ☐ Quadratica
- ☐ Logaritmica
- ☐ Costante
- ☒ Lineare

✗ **Non corretto** 0/1 Punti

10

Sia **h** la funzione *hash* utilizzata nell'implementazione di un dizionario in C mediante liste di trabocco. Sia **m** il numero di liste del dizionario ed **n** il numero di elementi. Quale di queste proprietà non è vera? [?]

- ☐ il dominio di **h** è l'insieme delle chiavi del dizionario
- ☐ **h** deve essere deterministica
- ☐ il dominio di **h** sono gli interi tra 0 e **m-1**

https://forms.office.com/ResponsePage.aspx?d=467fJGTxxLC3M1GxJ910questi_25-dfe58585ef8vUOFBMMV89MugSURVDMFZC... 4/7

03/05/23, 14:25

Prova Intermedia 2 - 2023-05-03

- ☒ il codominio di **h** sono gli interi tra 0 e **m-1**

Questo contenuto è creato dal proprietario del modulo. I dati raccolti verranno inviati al proprietario del modulo. Microsoft non è responsabile per la privacy o le procedure di sicurezza dei propri clienti, incluse quelle del proprietario di questo modulo. Non fornire mai la password.

Con tecnologia Microsoft Forms | [Privacy e cookie](#) | [Condizioni per l'uso](#) | [Accessibilità](#)



10:16

github.com

https://forms.office.com/pages/responsepage.aspx?Id=7fJGTx6LC2Y1LQaR9Dpobn_Z5-dhe58DA5aRvVUCF8WVVRMug3UxVDMFZC... 4/7

03/05/23, 14:25
Prova Intermedia 3 - 2023-05-03

Sia `a` una variabile intera a cui è stato assegnato il valore `n`.
 valore della variabile `c` al termine del seguente frammento di codice

```

int *b = &a;
int c = a*b;
a++;
    
```

☐ un indirizzo a partire da `&a`
☐ $2*(n+1)$
☒ $2*n$
☐ $2*n+1$

✗ Non corretto 0/1 Punti

8

Sia `n` un intero positivo, qual è la complessità temporale del seguente frammento di codice `c` ?

```

int i;
float *a = malloc(n*sizeof(float));
for (i = 0; i < n; i++)
    a[i] = i;
a = realloc(a, 2*n);
    
```

☐ lineare nel caso peggiore
☐ costante nel caso peggiore

https://forms.office.com/pages/responsepage.aspx?Id=7fJGTx6LC2Y1LQaR9Dpobn_Z5-dhe58DA5aRvVUCF8WVVRMug3UxVDMFZC... 5/7

03/05/23, 14:25
Prova Intermedia 3 - 2023-05-03

☐ quadratico nel caso peggiore
☒ lineare in tutti i casi

✓ Esatto 1/1 Punti

9

Sia `L` una lista concatenata implementata in C: ogni nodo contiene un intero

10:16

5G 73



Sia `n` un intero positivo ed `a` la stringa definita dal seguente frammento di codice, quanto vale `strlen(a)` ?

```
char *a = malloc(n*sizeof(char));
a[n/2] = '\0';
```

- ☒ al più `n/2`
- ☐ `n`
- ☐ `n/2`

https://forms.office.com/pages/responsepage.aspx?id=7FJGT0xLC2dYLCUR9Dpxb0h_Z5-df6585W5aR5vUCF8WVVRMug3UVDMFZC... 3/7

03/05/23, 14:25

Prova Intermedia 3 - 2023-05-03

☐ 0

✓ Esatto 1/1 Punti

6



Sia `a` una stringa di caratteri decimali, quanto vale la variabile `c` dal seguente frammento di codice ?

```
int i=0, n=0;

while( a[i] != '\0' ){
    n += a[i] - '0';
    i++;
}
```

- ☐ almeno la lunghezza di `a`
- ☐ al massimo la lunghezza di `a`
- ☒ la somma delle cifre in `a`
- ☐ la lunghezza di `a`

✓ Esatto 1/1 Punti

7



https://forms.office.com/pages/responsepage.aspx?id=7FJGT0xLC2dYLCUR9Dpxb0h_Z5-df6585W5aR5vUCF8WVVRMug3UVDMFZC... 4/7

03/05/23, 14:25

Prova Intermedia 3 - 2023-05-03

Sia `a` una variabile intera a cui è stato assegnato il valore `n`.

10:16

5G 73

✓ Esatto 1/1 Punti

3



Sia `L` una lista concatenata implementata in `C`: ogni nodo contiene un intero ed il puntatore al nodo successivo oppure `NULL` se è l'ultimo. `L` è il puntatore al primo nodo.

Si supponga che gli elementi in `L` siano ordinati dal più piccolo al più grande, qual è la complessità temporale della ricerca del massimo elemento?

- ☐ Quadratica
- ☐ Costante
- ☐ Logaritmica
- ☒ Lineare

✗ Non corretto 0/1 Punti

4



https://forms.office.com/pages/responsepage.aspx?id=7FJGT5xLC24YLC4R9Dpx0h1_ZG-dh092GAYsRvVUCF8WVRRMug3JAVDMFZC... 2/7

03/05/23, 14:25

Prova intermedia 3 - 2023-05-03

Sia consideri il seguente codice `C`

```
int a[123] = {0};
int *b = a;
```

Quale tra le seguenti affermazioni è vera

- ☒ `a[2]` è indefinito
- ☐ `sizeof(a) == sizeof(b)`
- ☐ `sizeof(a) > sizeof(b)`
- ☐ `sizeof(a) < sizeof(b)`

✓ Esatto 1/1 Punti

5



Sia `n` un intero positivo ed `a` la stringa definita dal seguente frammento di codice, quanto vale `strlen(a)`?

```
char *a = malloc(n * sizeof(char));
a[n/2] = '\0';
```

10:16

github.com

Prova intermedia 3 - 2023-05-03

Rispondere correttamente ad almeno 6 domande

Punti: 6/10

Esatto
1/1 Punti

1

Si consideri la struttura dati lista (o *array dinamico*) di n elementi come definita a lezione. Qual è la complessità temporale dell'operazione di inserimento di un nuovo elemento in posizione p ?

- ☐ costante nel caso medio
- ☐ lineare in p nel caso peggiore
- ☐ costante nel caso peggiore
- ☒ lineare in $n-p$

Non corretto
0/1 Punti

2

Si consideri un dizionario implementato in C mediante liste di trabocco. Sia n il numero di elementi nel dizionario ed m il numero di liste. Quale delle seguenti affermazioni deve essere vera affinché il dizionario sia efficiente?

- ☒ n/m deve essere costante

03/05/23, 14:25

Prova intermedia 3 - 2023-05-03

- ☐ la dimensione di ogni lista deve essere circa n/m
- ☐ la metà delle liste deve contenere meno di 4 elementi
- ☐ la funzione *hash* deve essere casuale

Esatto
1/1 Punti

3

Sia L una lista concatenata implementata in C: ogni nodo contiene un intero ed il puntatore al nodo successivo oppure `NULL` se è l'ultimo. $head$ è il puntatore al primo nodo.

10:11

5G 74

← DOC-20250511...   

1

(-/1 punti)

Sia a una lista concatenata contenente n interi ordinati in modo crescente, qual è il costo computazionale della ricerca di una chiave k in a ?

- ☒ costante
- ☐ ordine di $\log(n)$
- ☐ lineare

2

(1/1 punti)

Cosa viene stampato dal seguente codice Python?

```
a = ['zero', 'one', 'two', 'three', 'four', 'five', \
    'six', 'seven', 'eight', 'nine', 'ten' ]
d = {}
for i in range(len(a)):
    d[a[i]] = i

print( a[d['one']] + 3 + d[a[4]] )
```

- ☒ eight ✓
- ☐ 8
- ☐ Un errore

3


(0/1 punti)

Esercizio 6

a e b sono due liste concatenate contenenti interi e d un dizionario, inizialmente vuoto, implementato con liste di trabocco. Gli elementi di d sono coppie (k, v) dove la chiave k è di tipo intero e la chiave v è di tipo puntatore. Vengono eseguite le seguenti operazioni:

- per ogni elemento x di a , la coppia (x, NULL) viene inserita in d ;
- per ogni elemento x di b , la coppia (x, NULL) viene inserita in d .

Se a contiene n elementi e b ne contiene m , quanti elementi contiene d ?

- ☒ almeno $n+m$
- ☐ $\max(n,m)$
- ☐ meno di $n+m+1$ ✓



4

(1/1 punti)

Sia a una stringa qual è il risultato di `enigma(a)`?

10:12

5G 74

- ☐ una lista
- ☐ un intero
- ☐ un errore

8

(1 punto)

Sia a una stringa qual è il risultato di `enigma(a)`?

```
def enigma(x):
    if x == '':
        return 1
    else:
        return 1+enigma(x[1:])
```

- ☐ `len(a)+1`
- ☐ una stringa formata da `len(a)` '1'
- ☐ `len(a)`

9

(1 punto)

Sia a una stringa di lunghezza $n > 1$, cosa viene stampato dal seguente frammento di codice C?

```
int n = strlen(a);
char *b = a+n/2;
*b = '\0';
printf("%ld\n", strlen(a)-strlen(b));
```

- ☐ un numero > 0
- ☐ un numero < 0
- ☐ 0

10

(1 punto)

a e b sono due liste concatenate contenenti interi e d un dizionario, inizialmente vuoto, implementato con liste di trabocco. Gli elementi di d sono coppie (k, v) dove la chiave k è di tipo intero e la chiave v è di tipo puntatore. Vengono eseguite le seguenti operazioni:

- per ogni elemento x di a , la coppia (x, NULL) viene inserita in d ;
- per ogni elemento x di b , la coppia (x, NULL) viene inserita in d .

Se a contiene n elementi e b ne contiene m , quanti elementi contiene d ?

- ☐ almeno $n+m$
- ☐ $\max(n,m)$
- ☐ meno di $n+m+1$

10:12



7

(1 punto)

Sia n un intero positivo ed f definita come segue, qual è il risultato di $f(n)$?

```
def f(n):
    a = list(range(n))
    a.append(list(range(n, 2*n)))
    a += list(range(2*n, 3*n))
    return a[n]*a[n+1]
```

- ☐ una lista
- ☐ un intero
- ☐ un errore

8

(1 punto)

Sia a una stringa qual è il risultato di $\text{enigma}(a)$?

```
def enigma(x):
    if x == '':
        return 1
    else:
        return 1+enigma(x[1:])
```

- ☐ $\text{len}(a)+1$
- ☐ una stringa formata da $\text{len}(a)$ '1'
- ☐ $\text{len}(a)$

9

(1 punto)

Sia a una stringa di lunghezza $n > 1$, cosa viene stampato dal seguente frammento di codice C?

```
int n = strlen(a);
char *b = a+n/2;
*b = '\0';
printf("%ld\n", strlen(a)-strlen(b));
```

- ☐ un numero > 0
- ☐ un numero < 0
- ☐ 0

10

(1 punto)

a e b sono due liste concatenate contenenti interi e d un dizionario, inizialmente vuoto, implementato con liste di trabocco. Gli elementi di d sono coppie (k, v) dove la chiave k è di tipo intero e la chiave v è di tipo puntatore. Vengono eseguite le seguenti operazioni:



10:12

5G 74

1

(1 punto)

Si consideri la seguente funzione C:

```
void f(int *x, int p){
    *(a+p) = 2*p;
}
```

Sia a un array di $n > 0$ interi tale che $a[i] = i$ per ogni indice i e sia $0 \leq k < n$. Dopo l'invocazione di $f(a, k)$ qual è il valore di $a[k]$?

- ☐ indefinito
- ☐ $2k$
- ☐ k

2

(1 punto)

Qual è il costo computazionale della seguente funzione C?

```
void f(int n){
    int i, j;
    for(i = 0; i < n; i++){
        j = i;
        while( j < n ){
            j++;
        }
        while( j > 0 ){
            j--;
        }
    }
}
```

- ☐ n^2
- ☐ n
- ☐ n^3

3

(1 punto)

Sia a una lista concatenata contenente n interi ordinati in modo crescente, qual è il costo computazionale della ricerca di una chiave k in a ?

- ☐ lineare
- ☐ costante
- ☐ ordine di $\log(n)$

4

(1 punto)

`BubbleSort()` è una funzione Python che implementa la versione più

7

(1/1 punti)

Sia a una stringa di lunghezza $n > 1$, cosa viene stampato dal seguente frammento di codice C?

```
int n = strlen(a);
char *b = a+n/2;
*b = '\0';
printf("%ld\n", strlen(a)-strlen(b));
```

- ☐ un numero < 0
- ☐ 0
- ☒ un numero > 0 ✓

8

(1/1 punti)

Si consideri la seguente funzione C:

```
void f(int *x, int p){
    *(a+p) = 2*p;
}
```

Sia a un array di $n > 0$ interi tale che $a[i] = i$ per ogni indice i e sia $0 \leq k < n$. Dopo l'invocazione di $f(a, k)$ qual è il valore di $a[k]$?

- ☐ k
- ☒ $2k$ ✓
- ☐ indefinito

9

(1/1 punti)

Sia n un intero positivo e $0 \leq k < n$, qual è il valore di $\text{len}(a[k])$?

```
a = ['x']
for i in range(n):
    a.append('x'+a[-1])
```

- ☐ $k-1$
- ☐ k
- ☒ $k+1$ ✓

10

(1/1 punti)

Sia n un intero positivo ed f definita come segue, qual è il risultato di $f(n)$?

```
def f(n):
    a = list(range(n))
    a.append(list(range(n, 2*n)))
    a += list(range(2*n, 3*n))
    return a[n]*a[n+1]
```

8-10/10



4

(1/1 punti)

Sia a una stringa qual è il risultato di `enigma(a)`?

```
def enigma(x):
    if x == '':
        return 1
    else:
        return 1+enigma(x[1:])
```

- ☐ `len(a)`
- ☒ `len(a)+1` ✓
- ☐ una stringa formata da `len(a)` "1"

5

(1/1 punti)

Qual è il costo computazionale della seguente funzione C?

```
void f(int n){
    int i, j;
    for(i = 0; i < n; i++){
        j = i;
        while( j < n ){
            j++;
        }
        while( j > 0 ){
            j--;
        }
    }
}
```

5-6/10



- ☐ n
- ☒ n^2 ✓
- ☐ n^3

6

(1/1 punti)

`BubbleSort()` è una funzione Python che implementa la versione più ottimizzata dell'algoritmo *bubble-sort*: prende in input una lista di interi e la modifica ordinando i suoi elementi in modo crescente. Sia a una lista di n interi, qual è il costo computazionale (caso peggiore) della seguente funzione?

```
def MultSort(a, n):
    for i in range(n):
        BubbleSort(a)
```

- ☐ Lineare
- ☒ Quadratico ✓
- ☐ Cubico

7

(1/1 punti)

Sia a una stringa di lunghezza $n > 1$, cosa viene stampato dal



6

La lista **L** contiene **n** parole ordinate in modo alfabetico. Sia **p** una stringa. Qual è il costo del miglior algoritmo che ritorni la lista di parole di **L** che abbiano come prefisso la stringa **p**? (1 Point)

- ☐ $O(\log n)$ tempo, $O(1)$ spazio
- ☐ $O(n \log n)$ tempo, $O(1)$ spazio
- ☒ $O(n)$ tempo, $O(1)$ spazio ✓
- ☐ $O(n \log n)$ tempo, $O(n)$ spazio
- ☐ $O(\log n)$ tempo, $O(\log n)$ spazio
- ☐ $O(n)$ tempo, $O(n)$ spazio



Sia `a` una lista di interi, cosa restituisce `max(a, key=str)` ?

- ☐ L'intero più grande
- ☐ L'intero con più cifre 
- ☒ Se c'è, un intero che inizia con 9 ✓
- ☐ Viene generato un **`TypeError`**
- ☐ Una stringa



6

La lista **L** contiene **n** parole ordinate in modo alfabetico. Sia **p** una stringa. Qual è il costo del miglior algoritmo che ritorni la lista di parole di **L** che abbiano come prefisso la stringa **p**? (1 Point)

- ☐ $O(\log n)$ tempo, $O(1)$ spazio
- ☐ $O(n \log n)$ tempo, $O(1)$ spazio
- ☒ $O(n)$ tempo, $O(1)$ spazio ✓
- ☐ $O(n \log n)$ tempo, $O(n)$ spazio
- ☐ $O(\log n)$ tempo, $O(\log n)$ spazio
- ☐ $O(n)$ tempo, $O(n)$ spazio



Sia `a` una lista di interi, cosa restituisce `max(a, key=str)` ?

- ☐ L'intero più grande
- ☐ L'intero con più cifre 
- ☒ Se c'è, un intero che inizia con 9 ✓
- ☐ Viene generato un **`TypeError`**
- ☐ Una stringa



Si consideri il seguente frammento di codice C, quale tra le seguenti affermazioni è vera?

```
int *f( int *k ){
    *k = 3*k;
    return k;
}

void main(){
    int m,n;

    /*
     * ...
     * Operazioni in cui viene assegnato un valore a n
     * ...
     * */

    m = *f(&n);
}
```

- ☐ n e m sono uguali ✓
- ☐ m è un array
- ☐ n è 3 volte m
- ☐ m è 3 volte n
- ☐ *n è 3 volte *m
- ☒ n ed m puntano allo stesso indirizzo

Cosa succede eseguendo il seguente programma?

```
#include <stdio.h>
#include <string.h>

struct str {
    char *a;
    int n;
};
typedef struct str str;

str new_str(){
    str s;
    char x[] = "malloc";
    s.a = x;
    s.n = strlen(x);
    return s;
}

int main(){
    str b = new_str();
    printf("%s\n", b.a);
    return 6;
}
```

- ☐ Stampa l'intero 6
- ☒ Stampa la stringa malloc
- ☐ Errore in compilazione
- ☐ Un ciclo infinito
- ☐ Qualcosa di indefinito

Sia **a** una lista di interi, in quale casi tra quelli elencati la seguente espressione viene valutata **True** ?

$$2 * \text{sorted}(a) \leq a$$



- ☐ Se **a** è non ordinata
- ☐ Mai
- ☐ Sempre
- ☒ Solo se **a** è vuota
- ☐ Solo se **a** è ordinata

Siano **x** e **y** due variabili di tipo **int**, in quale caso, fra i seguenti, il valore di $(x*y)/2$ sarebbe sempre uguale a $(x*y)//2$?

- ☐ Sempre
- ☒ Mai
- ☐ Se **x** valesse **1**
- ☒ Se **x** valesse **y+1**
- ☐ Se **x** e **y** fossero primi

L'operatore `//` calcola la divisione intera tra i suoi operandi. Se **n** fosse un intero, $n/2$ sarebbe uguale a $n//2$ solo se **n** fosse pari. Tra le opzioni offerte, $x*y$ è pari sono nel caso in cui **y** è il consecutivo di **x**. Infatti se **x** fosse pari allora anche il prodotto con **y** sarebbe divisibile per 2. Se **x** fosse dispari, **y** sarebbe pari e quindi anche il prodotto con **x**.

Programmazione dei Calcolatori con Laboratorio

Test preliminare

Rispondere a tutte le domande elencate barrando il cerchietto corrispondente alla risposta esatta.

-
1. Qual è il valore di $i - j$ al termine del seguente frammento di codice.

```
int i = 0, j = 8;
while( i <= j ){
    i = i+1; j = j-1;
}
```

Risposte

- ☐ - 1
☐ 1
☐ 2

-
2. Quante volte viene eseguito il test del **while** nel seguente frammento di codice.

```
int i = 0, j = n/2;
while( i < n/2 && j < n ){
    if ( a[i] < a[j] ){
        i = i+1;
    } else {
        j = j+1;
    }
}
```

Risposte

- ☐ al massimo
n/2+1 volte
☐ almeno
n/2 volte
☐ almeno
n volte



3



(0 punti)

Si consideri il problema di ordinare una sequenza contenente $2n + 3$ interi maggiori di $4n + 4$ e minori di $6n + 8$. Sia A un algoritmo efficiente che risolve il problema, qual è l'ordine di grandezza del suo costo computazionale?

☒ n ✓

☐ $n \log(n)$

☐ n^2

☐ Costante

Intervallo $\mathbb{R}$
DEFINIZIONE
da $4n+4$ a $6n+8$

(1 punto)

Si consideri la seguente funzione C:

```
void f(int *x, int p){
    *(a+p) = 2*p;
}
```

Sia a un array di $n > 0$ interi tale che $a[i] = i$ per ogni indice i e sia $0 \leq k < n$. Dopo l'invocazione di $f(a, k)$ qual è il valore di $a[k]$?

- ☐ indefinito
- ☐ $2k$
- ☐ k

2

```
}
    i = i+1;
}
```

3. Se $a = \{4, 9, 2, 5, 1\}$, qual è il valore di $\text{incognito}(a, 0, 4)$?

```
int incognito( int a[], int i, int n ){
    if ( i == n )
        return i;
    if ( a[i] < incognito(a, i+1, n) )
        return a[i];
    else
        return incognito(a, i+1, n );
}
```

Risposte

- ☐ 1
- ☐ 2
- ☐ 4

4. Cosa viene stampato dal seguente frammento di codice?

```
char a[] = "abcdefghijklmno";
```

Risposta